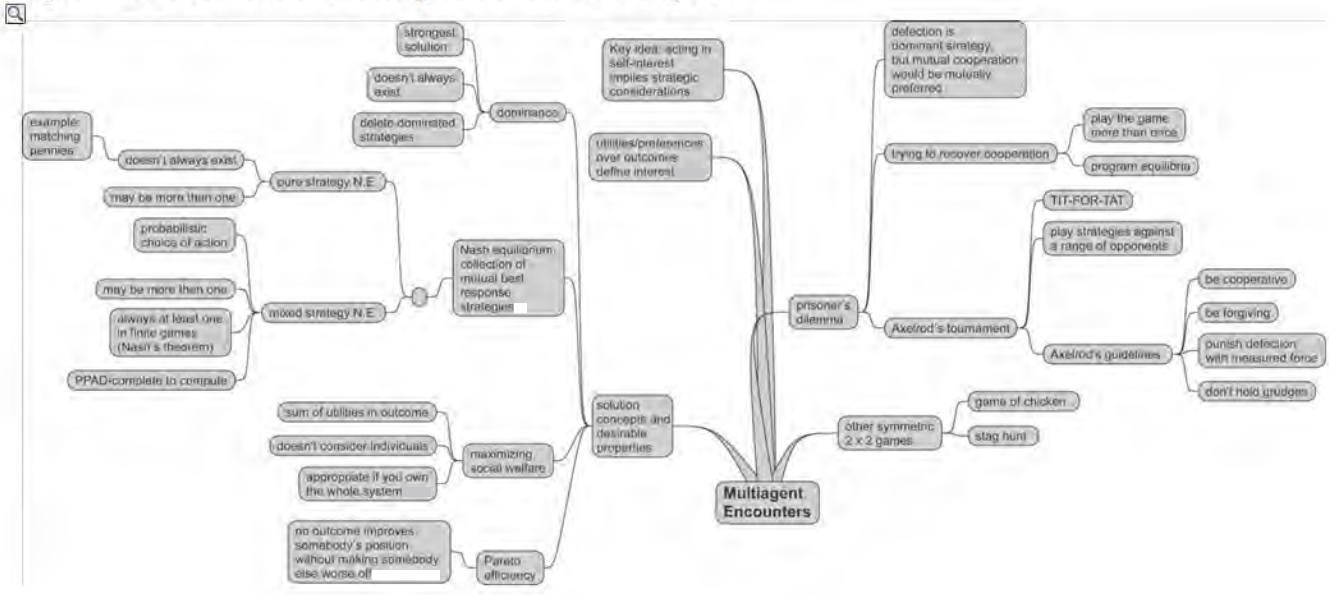
cooperation. But if noise causes one of them to misinterpret defection as cooperation, then this agent will retaliate to the perceived defection with defection. The other agent will retaliate in turn, and both agents will defect, then retaliate, and so on, losing significant utility as a consequence. Interestingly, cooperation can be restored if further noise causes one of the agents to misinterpret defection as cooperation – this will then cause the agents to begin cooperating again! [Axelrod, 1984] is recommended as a point of departure for further reading; [Mor and Rosenschein, 1995] provides pointers into the prisoner's dilemma literature; a collection of Axelrod's more recent essays was published as [Axelrod, 1997].

Class reading: [Axelrod, 1984]. This is a book, but it is quite easy going, and most of the important content is conveyed in the first few chapters. Apart from anything else, it is beautifully written, and it is easy to see why so many readers are enthusiastic about it. However, read it in conjunction with Binmore's critiques, cited in the *Notes and Further Reading*.

Figure 11.5: Mind map for this chapter.



Chapter 12 Making Group Decisions

In the previous chapter, we looked at the general setting of multiagent encounters, in which it was assumed that such encounters have a game-like character, and participants will act strategically in order to select the action that they believe will lead to the best possible outcome for themselves. In this chapter, we study a class of protocols specifically intended for *making group decisions*. This is the domain of *social choice theory*, or, a bit more informally, *voting theory*. The voting protocols we will consider have a strategic flavour, because the agents participating in the protocol will have preferences over the possible outcomes of voting, and they will take into account their own preferences as well as those of others when making their decisions about how to vote, in order to try to bring about their most preferred outcome. We will see that, when studied from a computational perspective, such voting protocols have some rather intriguing properties.

SOCIAL CHOICE

VOTING THEORY

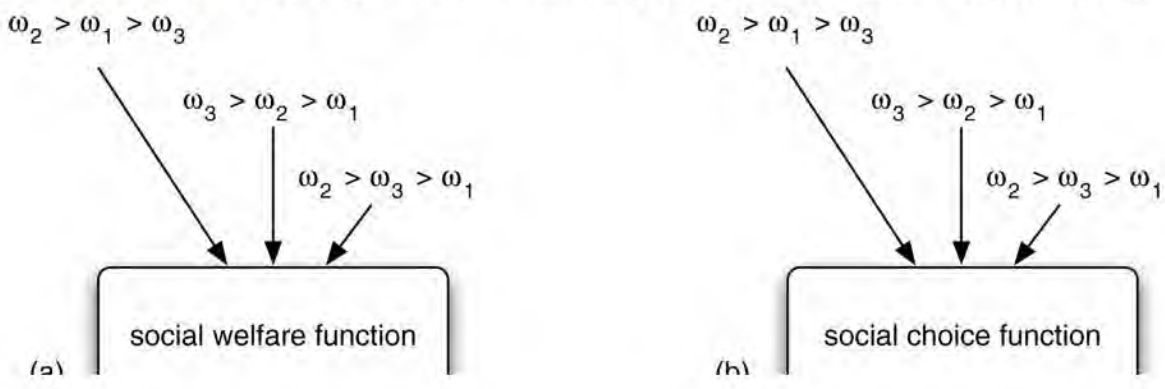
12.1 Social Welfare Functions and Social Choice Functions

The basic setting we are going to consider is as follows. We have a set $Ag = \{1, \ldots, n\}$ of agents, who in this chapter we will often refer to as the *voters*. These are the participants in the group decision-making process. It is of course assumed that Ag is finite, but we will also usually assume, without stating it, that Ag contains an odd number of voters. The simple reason for this is that it tends to eliminate the possibility of *ties* in elections (in many real-world voting situations, the electorate is explicitly engineered so that $|Ag|$ is odd, specifically for this reason).

Voters will be making group decisions with respect to a set $\Omega = \{\omega_1, \omega_2, \ldots\}$ of possible *outcomes* or *candidates*. Usually, it is assumed that this set is finite. The voters will be aiming to *rank* or *order* these candidates; sometimes they will be aiming simply to *choose one* (the top ranked candidate). In a real-world setting, for example, Ω might be the set of political candidates in an election. Sometimes, we make some special assumptions about the set of outcomes, depending on the type of scenario that we are modelling. For example, if we assume $|\Omega| = 2$, then we say the setting is one of a *pairwise election*. If $|\Omega| > 2$, then we say that the setting is a *general* voting scenario.

PAIRWISE ELECTION

Figure 12.1: Social welfare functions and social choice functions.



Each voter will have preferences over Ω, and, for the purposes of this chapter, we will assume that these preferences are defined simply as an ordering over the set of possible outcomes Ω. For example, agent i's preference ordering might be $(\omega_2, \omega_3, \omega_1)$, meaning that agent i prefers ω_2 over ω_3, and prefers ω_3 over ω_1. We will write $\omega_1, \ldots, \omega_n$ to denote the preference orders of agents $1, \ldots, n$, and we will write $\omega \succ_i \omega'$ to mean that outcome ω is ranked above outcome ω' in agent i's preference order ω_i. We denote the set of all preference orderings over outcomes Ω by $\Pi(\Omega)$.

Now, the fundamental problem addressed in social choice theory is that agents might naturally have *different* preference orders: so, *given a collection of preference orders, one for each agent, how do we combine these to derive a group decision*?

PREFERENCE AGGREGATION

A *social welfare function* takes the voter preferences and produces a *social preference order*: essentially, a ranking of the candidates, from most preferred down to least preferred (see Figure 12.1(a)). Formally, a social welfare function can be understood as a function:

$$f : \underbrace{\Pi(\Omega) \times \cdots \times \Pi(\Omega)}_{n \text{ times}} \rightarrow \Pi(\Omega).$$

We will sometimes use $\succ^*$ to refer to the outcome of a social welfare function; that is, we will write $\omega \succ^* \omega'$ to mean that ω is ranked above ω' in the social outcome.

Sometimes, we will be concerned with slightly simpler settings, in which we are not concerned with obtaining an entire ordering, but just one of the possible candidates (Figure 12.1(b)). We will use the term *social choice function* to refer to such a mapping:

$$f : \underbrace{\Pi(\Omega) \times \cdots \times \Pi(\Omega)}_{n \text{ times}} \rightarrow \Omega.$$

In what follows, we will collectively refer to social welfare functions and social choice functions as *voting procedures*. Next, we will look at some examples of voting procedures.

12.2 Voting Procedures

There is a huge literature on voting procedures, and we cannot hope to do justice to it all here. We give some examples of the key voting procedures, and the most important from the point of view of multiagent systems.

12.2.1 Plurality

We will start by considering arguably the simplest and surely the best-known voting procedure: *plurality*. The plurality procedure is most commonly used to select a single candidate, rather than produce a ranked list of candidates, although the idea straightforwardly

generalizes. The basic idea is that every voter submits their preference order; we then count how many times each outcome is ranked first in a preference order. The winner is the outcome that appears first in the preference orders the largest number of times. (Since we are only concerned with first-place positions, we can of course make the procedure simpler for voters by only requiring them to indicate their first-ranked candidate, rather than their entire preference order: this is of course exactly what we are doing when we make a mark on a ballot.)

PLURALITY VOTING

The main advantages of plurality are that it is straightforward to implement, and it is easily understood by voters. If we only have two outcomes to choose between, then plurality is just *simple majority voting*: in this case, we simply ask candidates to select one of the two outcomes, and the one that gets the majority of votes is the winner. However, some anomalous properties start to appear if we have more than two candidates in plurality voting. Let us consider an example, to illustrate one of these paradoxes.

SIMPLE MAJORITY VOTING

The example is roughly based on the political scene in the UK. While there are many political parties in the UK, only three have had any substantial influence in the past two decades: the Labour Party, the Liberal Democrats, and the Conservative Party. The Labour Party are (historically at least) a left-wing party, while the Liberal Democrats are centre-left, and the Conservative Party are right wing. The left- and right-wing parties dominate the political agenda, with the centre party having relatively little support. The balance of power is rather slim between left and right wings: neither side strongly dominates. Most voters with socialist or liberal tendencies will vote for the Labour Party, and would then prefer the Liberal Democrats over the Conservative Party; however, a minority would prefer the Liberal Democrats over the Labour Party, but would still prefer the Labour Party over the Conservatives. Finally, right-wing voters would typically prefer the Conservative Party over the Liberal Democrats over the Labour Party. Given this background, a typical situation in a UK general election is to have three candidates, ω_L, ω_D, and ω_C, representing the Labour Party, Liberal Democrats, and Conservative Party, respectively. About 44% of the voters would be conventionally left-wing, with the following preference order.

$$\text{left-wing voter: } \omega_L \succ \omega_D \succ \omega_C.$$

However, as indicated above, a minority of liberally inclined voters prefer the more moderate Liberal party – about 12% of voters would have the following preference order.

$$\text{center-left voter: } \omega_D \succ \omega_L \succ \omega_C.$$

Finally, the remaining 44% of voters would be right-wing, with the following preference profile.

$$\text{right-wing voter: } \omega_C \succ \omega_D \succ \omega_L$$

With the plurality voting procedure, the Conservative Party candidate ω_C wins, with 44% of the voters ranking this candidate above the others. But for 56% of voters – a *majority* of the electorate – candidate ω_C was the *least preferred option!* It is important to note that this is not an artificial example – situations like this occur all the time in politics, and the consequences

an artificial example – situations like this occur all the time in politics, and the consequences affect us directly.

In recent years, this has given rise to a phenomenon known as *tactical voting*. Suppose you are in a voting district that leans more towards the Conservative candidate, with Labour running third in terms of percentage support. You personally favour the Labour candidate, with a Liberal Democrat candidate second and Conservative third. With simple plurality voting, placing the Labour candidate first in your ranking (i.e. voting for that candidate) may well be wasted – your voting district is too right wing to elect a Labour candidate. If you instead rank the Liberal candidate first, you might be able to get the Liberal candidate elected, and bring about an outcome that you prefer over the election of a Conservative candidate. Notice that *you are not representing your preferences truthfully here:* you do not honestly rank the Liberal candidate first. What you are doing is *strategically misrepresenting your preferences* in order to bring about a more preferred outcome.

TACTICAL VOTING

STRATEGIC MANIPULATION

The example above hints at some more fundamental problems with voting procedures in general. Suppose we have three outcomes, $\Omega = \{\omega_1, \omega_2, \omega_3\}$, and three voters, $Ag = \{1, 2, 3\}$, with preferences as follows.

$$\begin{aligned} \omega_1 &\succ_1 \omega_2 \succ_1 \omega_3 \\ \omega_3 &\succ_2 \omega_1 \succ_2 \omega_2 \\ \omega_2 &\succ_3 \omega_3 \succ_3 \omega_1 \end{aligned} \tag{12.1}$$

In this case, the result is tied with plurality voting: there is no winner, since each outcome is ranked first exactly once. But the situation is actually worse than this. Consider the merits of each outcome in turn:

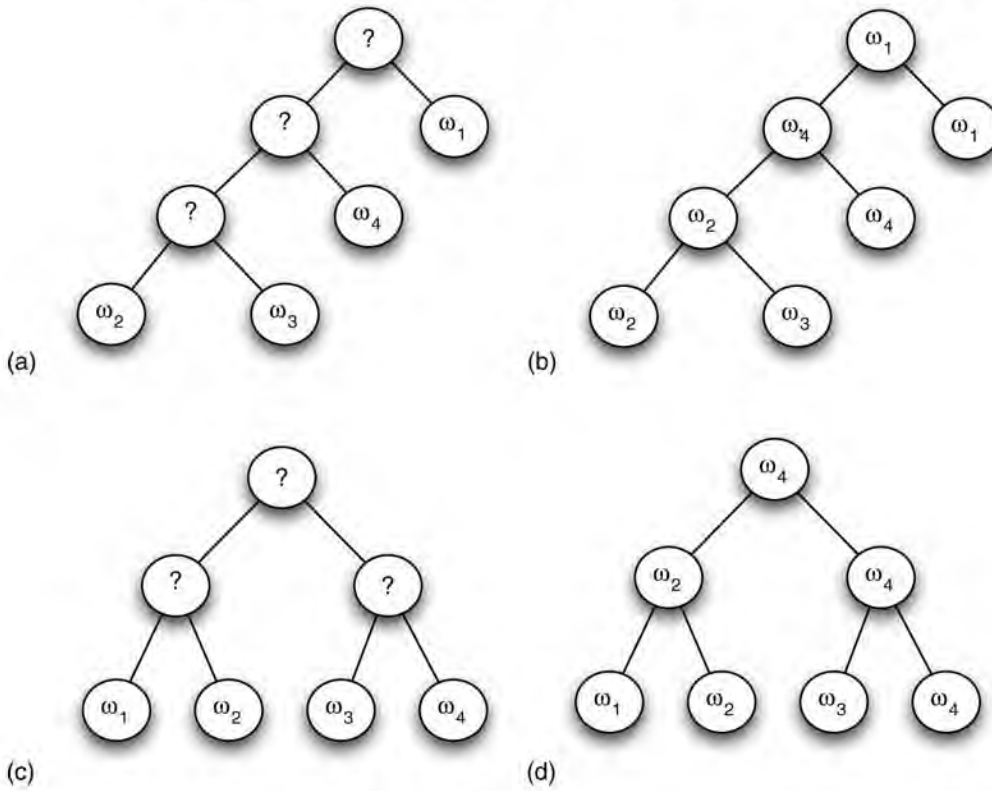
- Suppose we select candidate ω_1. But $\frac{2}{3}$ of the electorate – a large majority – would prefer ω_3 over ω_1.
- So, motivated by this argument, we suggest choosing ω_3. But again, $\frac{2}{3}$ of the electorate would prefer another specific outcome, ranking ω_2 above ω_3.
- So, suppose in a last desperate bid to get an outcome, we meekly suggest outcome ω_2. But again, $\frac{2}{3}$ of the electorate complain: they would prefer ω_1 over ω_2.

Thus for every possible candidate, there is another candidate who is preferred by $\frac{2}{3}$ of the electorate! This is perhaps the canonical example of what is known as *Condorcet's paradox*, which is one of the simplest and most profound difficulties with voting procedures. The significance of Condorcet's paradox is this: while it seems intuitively inevitable that in a democracy we cannot hope to choose an outcome that keeps *every* voter happy, Condorcet's paradox tells us that there are scenarios in which *no matter which outcome we choose, a*

majority *of voters will be unhappy with the outcome chosen.*

CONDORCET'S PARADOX

Figure 12.2: Sequential majority elections can be arranged into linear order of ballots (a) or a tree (b). As shown in (c) and (d), the outcome can be different, depending on how the candidates are placed in the voting agenda.



12.2.2 Sequential majority elections

Simple and popular though it undoubtedly is, plurality clearly has some problems, not the least being that it can select an outcome even though another outcome would be preferred by a majority of the electorate. Let us consider some common variants of plurality voting, and see how these variants fare in comparison. Perhaps the most natural variation is to consider organizing not just one, but a series of elections. The idea is that a pair of outcomes will face each other in a *pairwise* election, and the winner will then go on to the next election. For example, suppose we are voting over four outcomes, ω_1, ω_2, ω_3 and ω_4. We might choose the following order, or *agenda* for the election:

ELECTION AGENDAS

$$\omega_2, \omega_3, \omega_4, \omega_1.$$

Thus, the first election will be between ω_2 and ω_3; the winner of this election will go on to face ω_4; then the winner of this election will face ω_1. The winner of this final election is declared the overall winner. Figure 12.2(a) illustrates the idea, with Figure 12.2(b) showing a possible outcome of this voting order. Of course, as Figure 12.2(c) suggests, the elections need not be arranged into a linear order: another possibility is to have a *balanced binary tree* of elections. In the case illustrated in Figure 12.2c, the winner of the election between ω_1 and

ω_2 will go on to face the winner of the election between ω_3 and ω_4, and the winner of this third election is declared the overall winner.

BINARY VOTING TREE

Figure 12.2d indicates one key problem with this kind of voting procedure: the final outcome selected may depend not just on voter preferences, but *on the order in which the candidates come up for election*, i.e. the election agenda. This does not seem desirable, because it suggests that, if the agenda is selected at random, then randomness has some part to play in a democratic process, while if the agenda is chosen otherwise, then it opens up the possibility of the election being *manipulated* by an electioneer.

To illustrate this point further, let us introduce some terminology. We create a directed graph, called a *majority* graph, from voter preferences. The idea is that the nodes in the graph will correspond to outcomes Ω, and *there will be an edge in the graph from outcome ω to outcome ω' if a majority of voters rank ω above ω', i.e. if ω would beat ω' in a direct competition*. To keep things simple, let us assume that there are an odd number of voters, so that there are no ties. The majority graph we construct in this way will have some special properties:

- the majority graph is *complete*: for any two outcomes ω_i and ω_j, we must have *either* ω_i defeats ω_j or ω_j defeats ω_i
- the majority graph is *asymmetric*: if ω_i defeats ω_j, then it cannot be the case that ω_j defeats ω_i
- the majority graph is *irreflexive*: an outcome will never defeat itself.

A graph with these properties is called a *tournament* [Moon, 1968]. One way to think of the majority graph is as a *succinct representation of voter preferences*. From a computational point of view, it may not be desirable or practical to list the complete preference orders for all voters, and we can summarize a lot of the information contained in those preference orders in a majority graph.

TOURNAMENTS

Now, suppose we are going to organize a linear sequential majority election for three candidates: ω_1, ω_2, and ω_3, with 99 voters, who have preferences as follows:

- 33 voters have preferences of the form $\omega_1 \succ_i \omega_2 \succ_i \omega_3$
- 33 voters have preferences of the form $\omega_3 \succ_i \omega_1 \succ_i \omega_2$
- 33 voters have preferences of the form $\omega_2 \succ_i \omega_3 \succ_i \omega_1$.

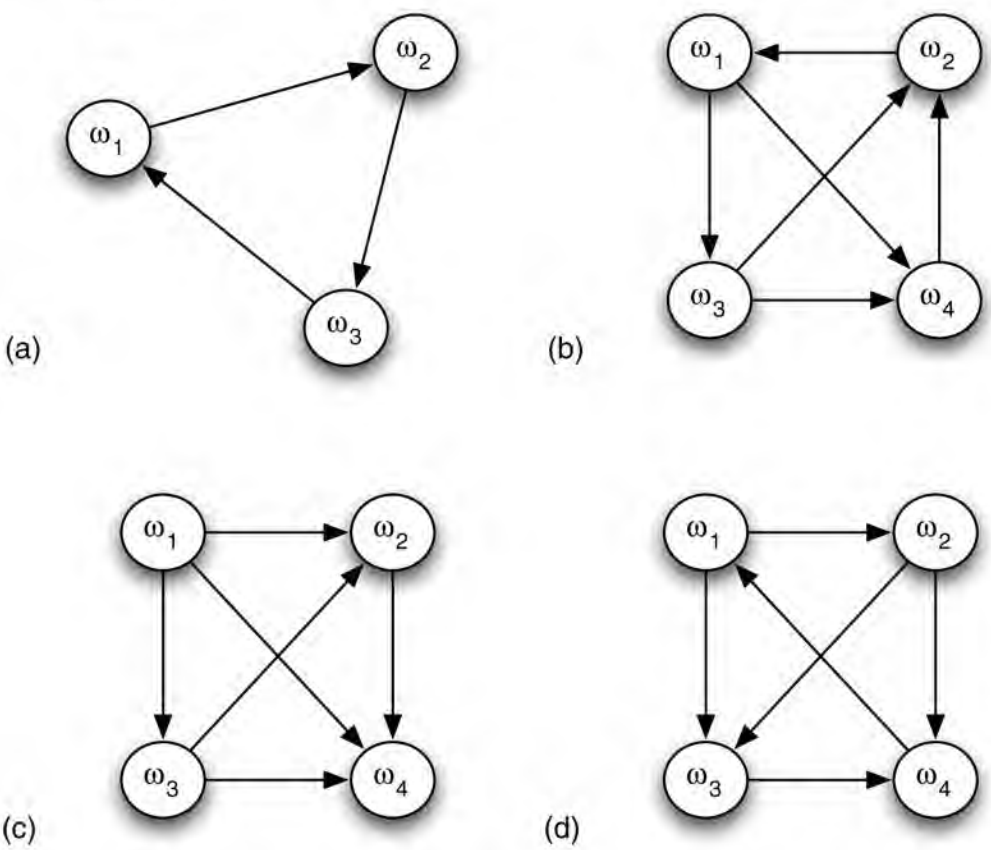
The majority graph for this situation is easy to compute: it is shown in Figure 12.3a. The graph is a simple cycle: starting at any node, there is a single path, which leads eventually back to the same node. Given this majority graph, a remarkable fact becomes apparent: *we can fix an election agenda so that any of the three candidates will win!* For example, if we want candidate ω_1 to win, then the agenda ω_3, ω_2, ω_1 will do, since we know from the majority graph that ω_2 will beat ω_3, and then ω_1 will beat ω_2. If we want ω_3 to win, then the order ω_2, ω_1, ω_3 will do it: ω_1 will beat ω_2, and then ω_3 will beat ω_1. (What agenda will bring about

ω_1, ω_3 will do it. ω_1 will beat ω_2, and then ω_3 will beat ω_1. (What agenda will bring about outcome ω_2 winning?) Figure 12.3b gives another example of a majority graph in which every outcome is a possible winner (for example, the agenda ω_3, ω_2, ω_4, ω_1 will result in outcome ω_1 winning, while the agenda ω_1, ω_4, ω_2, ω_3 will result in ω_3 winning – which agendas will result in a win for ω_2 and ω_4?).

Let us define some terminology to make these ideas a bit more precise. An outcome is a *possible winner* if there is some agenda which would result in that outcome being the overall winner. An outcome is called the *Condorcet winner* if it is the overall winner for every possible agenda. Now, to determine whether an outcome ω_i is a possible winner, all we have to do is to check whether, for every other outcome ω_j, there is a path from ω_i to ω_j in the majority graph. It is computationally easy to determine whether a path exists from one node to another (this is just the 'graph reachability' problem [Papadimitriou, 1994, p. 3]), and so we can compute the answer to the question of whether a candidate is a possible winner very efficiently. Of course, checking whether a candidate is a Condorcet winner is also very easy: to check that ω_i is a Condorcet winner simply involves checking whether there is an edge from ω_i to every other node in the majority graph. Figure 12.3c shows a majority graph in which outcome ω_1 is a Condorcet winner.

CONDORCET WINNER

Figure 12.3: Majority graphs. In (a) and (b), every outcome is a possible winner. In (c), outcome ω_1 is a Condorcet winner.



The discussion so far make one important assumption: that we know exactly what the preferences of the voters are. However, in reality, we are not likely to have this information.

Instead, what we might have is partial information – probabilities that one candidate will defeat another, or partial models of voter preferences. Where we have such incomplete information about voter preferences, it seems that manipulation is computationally harder, although I caution that this is an area of ongoing research [Hazon et al., 2008a,b; Lang et al., 2007]. However, it turns out that some simple rules of thumb work surprisingly well for the manipulation of voting agendas [Hazon et al., 2008b]. For example, suppose you are trying to construct a voting order so as to give the best chance possible to an outcome ω_i: then rank all the candidates in order, from most likely to defeat ω_i down to least likely to defeat ω_i, with ω_i as the final candidate. This heuristic works because candidates appearing early in a linear voting order would have to face more elections in order to progress to a point where they face candidate ω_i.

It should now be clear from this discussion that democratic processes may not be quite as democratic as they appear: a manipulative committee chair can benefit from putting up candidates for election in a particular order, thereby favouring one of the candidates; voters, believing that their most favoured candidate has no chance in a simple majority election, can misrepresent their preferences in favour of another candidate with a better chance of winning. What we have witnessed thus far are simple examples of the *strategic manipulation* of social choice processes. We will return to the issue of strategic manipulation, and the role that computation plays in this, later in this chapter. However, before doing this, let us consider some other well-known voting procedures.

12.2.3 The Borda count

One conceptual problem with the voting procedures considered so far is that they 'ignore' a lot of the information represented in a preference order: they only consider top-ranked candidates, and collapse this information down into a simple majority decision. The *Borda count* takes into account all the information from a preference order. Suppose we have k candidates, i.e. $|\Omega| = k$. For each of these possible candidates, we have a numeric variable, counting the strength of opinion in favour of this candidate. Each voter preference order contributes to these counts as follows. If an outcome ω_i appears first in the preference order, then we increment the count for ω_i by $k - 1$; we then increment the count for the next outcome in the preference order by $k - 2$, and so on, until the final outcome in the preference order has its total incremented by 0. We proceed with this process until all preference orders have been considered, and then simply order the outcomes Ω by their count, largest first, down to smallest. If we are simply aiming to select a single candidate then the candidate with the largest count is chosen.

BORDA COUNT

12.2.4 The Slater ranking

The Slater rule is interesting because it considers the question of which social ranking should be selected as being the question of trying to find a consistent ranking (one that does not contain cycles) that is as close to the majority graph as possible – to *minimize the number of disagreements between the majority graph and the social choice*. To put it another way, imagine all possible orderings of the candidates; for each ordering, we measure how much that ordering would disagree with the information provided in the majority graph, that is, how

many edges in the graph would have to be 'flipped' in order to make the corresponding ordering consistent with the majority graph. We then ask which ordering minimizes this inconsistency measure. If a majority graph has no cycles, of course, then there is no difficulty in identifying a consistent ranking.

SLATER RULE

Consider the majority graph in Figure 12.3c. The graph has no cycles, and we can identify the following social choice as being acceptable:

$$\omega_1 \succ^* \omega_3 \succ^* \omega_2 \succ^* \omega_4.$$

In this ranking, every candidate ω_i that appears before a candidate ω_j in the ordering would beat candidate ω_j in a pairwise election according to the majority graph Figure 12.3c.

In contrast, consider Figure 12.3d. This majority graph has a cycle, and so whichever ordering we choose will disagree with the majority graph, in the sense that at least one candidate ω_i will be ranked above a candidate ω_j despite the fact that ω_j would beat ω_i in a pairwise election. For example, consider the following possible social choice ordering:

$$\omega_1 \succ^* \omega_2 \succ^* \omega_3 \succ^* \omega_4 \tag{12.2}$$

Here, outcome ω_1 appears before ω_4, despite the fact that ω_4 beats ω_1 according to the majority graph in Figure 12.3d. So, how inconsistent is ordering (12.2)? Well, to have no inconsistencies in the ordering, we would obviously have to flip the edge (ω_4, ω_1). In fact, this is the only edge that we would have to flip, and so we will say the cost of this order is 1.

Now consider the following rank:

$$\omega_1 \succ^* \omega_2 \succ^* \omega_4 \succ^* \omega_3 \tag{12.3}$$

Here, we would still have to flip the edge (ω_4, ω_1), but in addition, since ω_3 beats ω_4 according to the majority graph, we would also have to flip the edge (ω_3, ω_4). No other edges would need to be flipped. This gives (12.3) a cost of 2. To obtain the Slater ranking, we find out the cost of each ordering, and then choose one with the lowest cost. (In fact, (12.2) is the ordering that would be selected by the Slater ranking for this example.)

The Slater ranking is attractive because it takes seriously the idea of trying to come up with a solution that is 'optimal' in the sense that it is as close to the majority graph as possible. Unfortunately, as with so many combinatorial optimization problems, computing the Slater ranking is not easy: in fact, it is NP-hard [Conitzer, 2006b]. There are some techniques available to reduce the computational effort required to find the Slater ranking. For example, [Conitzer, 2006b] proposes a method based on identifying *similar candidates*. The idea is that two outcomes are similar if they defeat and are defeated by exactly the same outcomes. Given this, [Conitzer, 2006b] shows how multiple similar outcomes can be 'factored out' from consideration (essentially by considering them as if they were a single candidate), thereby reducing the overall search space. However, the fact that computing Slater rankings is NP-hard does call into question the usefulness of the approach in practical situations.

12.3 Desirable Properties for Voting Procedures

In the discussion so far, we have focused on specific examples of voting procedures, and where appropriate discussed their properties, both from a democratic and from a computational perspective. As we have seen, some voting procedures have properties that we might not be entirely comfortable with. A natural question, therefore, is whether there is any 'good' voting procedure – as we will see, the answer to this question, in a very precise sense, is 'no', and this *impossibility result* is probably the most famous result in social choice theory. But before we can get to this result, we need to be precise about what we mean when we say that a rule is 'good'. The obvious way to do this is to define the properties that a 'good' social welfare function would satisfy. In fact, there is an extensive literature on this subject, and we will here consider just some of the most important properties [Campbell and Kelly, 2002].

The Pareto condition

Recall from Chapter 11 that an outcome ω is said to be Pareto optimal, or Pareto efficient, if there is no other outcome that makes one agent better off without making another agent worse off. The Pareto condition for voting rules simply says that the preference order selected should be efficient in this sense: if every voter ranks ω_i above ω_j, then $\omega_i \succ^* \omega_j$. The Pareto condition is satisfied by plurality and Borda, but not by sequential majority elections.

PARETO CONDITION

The Condorcet winner condition

You will recall that an outcome is said to be a Condorcet winner if it would beat every other outcome in a pairwise election. Intuitively, it seems that a Condorcet winner is a very strong kind of winner indeed! The Condorcet winner condition says that if ω_i is a Condorcet winner, then ω_i should be ranked first (and so, in the case of social choice functions, is the outcome selected). Although this property seems a very obvious one, of the voting procedures we have discussed so far, only sequential majority elections satisfies it.

CONDORCET WINNER

Independence of irrelevant alternatives (IIA)

This condition seems a little complex at first, and is perhaps best explained by way of example. Suppose there are a number of candidates, including ω_i and ω_j, and voter preferences are such that $\omega_i \succ^* \omega_j$. Now, all of a sudden, you decide to change your preferences, but *not* about the relative ranking of ω_i and ω_j; you rank these the same as before. The *independence of irrelevant alternatives* (IIA) condition says that, in this case, we should still have $\omega_i \succ^* \omega_j$. To put it another way, the social ranking of ω_i and ω_j should depend only on the way that ω_i and ω_j are ranked in the preference orders – this is the only thing that should be taken into account when ranking ω_i and ω_j. As with the Condorcet winner property, surprisingly few protocols satisfy IIA: plurality, Borda, and sequential majority elections do not.

IIA

Dictatorships

The final property we will consider is not in fact a *desirable* property, but I hope you will agree that it is one that we should at least be aware of. The property is that of a voting procedure being a *dictatorship*. A social welfare function f is a dictatorship if for some voter i we have

DICTATORSHIPS

$$f(\omega_1, \ldots, \omega_n) = \omega_i. \tag{12.4}$$

Thus, the social welfare function defined in (12.4) takes as input a preference order for every agent, and then produces as output … the preference order of voter i! All other voters' preferences are simply ignored. Clearly, this does not seem like a very reasonable social welfare function – intuitively, we usually want a social welfare function that selects a preference ordering which, as closely as possible, reflects the preferences of the electorate. We will say a social welfare function is a *non-dictatorship* if it does not correspond to a rule of the form (12.4). Of course, plurality, Borda, and the Slater ranking are not dictatorships, but voting procedures of the form in (12.4) actually satisfy a surprising number of desirable properties. Specifically, dictatorships satisfy the Pareto condition and IIA.

12.3.1 Arrow's theorem

Above, we informally asked whether there was a 'good' social choice procedure; we can now make this precise, with respect to the conditions we have presented above. Given a list of such conditions, we can ask whether there exists any voting procedure that satisfies these conditions. As we will now see, a seminal result tells us that, basically, the answer to this question is 'no'. The result is called *Arrow's theorem*, after the man who first proved it. First, let us assume that we are not in a pairwise election setting, i.e. we have $|\Omega| > 2$. Now, let us say that a voting procedure is 'good' if it satisfies the Pareto condition and the independence of irrelevant alternatives condition. Now, to our basic question, 'Is there a "good" voting procedure?', the answer is yes. In fact, we have already seen such a social welfare function. But it wasn't plurality, or the Borda count, or the Slater ranking. It was the voting procedure defined in (12.4), that is, *a dictatorship!*

ARROW'S THEOREM

Well, this seems like a wise-guy answer to the question we asked, and so, somewhat irritated, we might ask another question, as follows. What *other* 'good' voting procedures are there, apart from dictatorships? Arrow's theorem is the answer to this question: there are in fact no other 'good' voting procedures – the *only* voting procedures satisfying the Pareto condition and independence of irrelevant alternatives are dictatorships. Let us put this another way. Suppose you try to define some conditions for voting procedures, which you think a 'democratic' voting procedure should satisfy, and you then define a voting procedure which satisfies these properties. Well, if your list of properties satisfies the Pareto condition and independence of irrelevant alternatives, then the procedure you defined is in fact nothing other than a dictatorship.

Arrow's theorem is, I am sure you will agree, profoundly troubling. It is troubling in much the

same way that the prisoner's dilemma in the preceding chapter was troubling. Our analysis of the one-shot prisoner's dilemma seemed to tell us that rational cooperation was impossible; a pessimistic interpretation of Arrow's theorem would be that a dictatorship is the best that we can hope for in a voting procedure. However, the situation is not quite so bleak. The obvious line of attack is to look at the way the result is set up, and, in particular, to ask whether the properties we identified are in fact really appropriate. Of the properties we identified, the least obvious one – and hence the most contentious one – is independence of irrelevant alternatives, and indeed this condition is probably the one that has drawn most attention. But Arrow's theorem towers above all efforts to define 'good' voting procedures, an implacable reminder that there are meaningful, fundamental limits to what can be done here.

From a computational point of view, interesting questions relating to Arrow's theorem relate to the *analysis* of voting procedures, when these procedures are represented in the form of *computer programs*. For example, suppose we define some particular decision-making procedure as a web-based tool for use in our organization. We might reasonably ask, in addition to whether the program works correctly, what social choice properties it satisfies. Some approaches to this problem are described in Chapter 17.

12.4 Strategic Manipulation

While Arrow's theorem is surely the best-known result in social choice theory, other results are almost as troubling. Perhaps the best known of these is the *Gibbard–Satterthwaite theorem*, which relates to the circumstances under which a voter can benefit from strategically misrepresenting their preferences, as we described, in the context of the plurality procedure, in Section 12.2.1.

GIBBARD–SATTERTHWAITE THEOREM

Recall that a social choice function takes as input a preference order for each voter, and gives as output a selected candidate.

$$f : \underbrace{\Pi(\Omega) \times \cdots \times \Pi(\Omega)}_{n \text{ times}} \to \Omega.$$

Each voter has, of course, their own 'true' preference profile, though this will be private information: my real preferences are not visible to you, nor are yours to me.

Now, a voter is free to declare *any* preference profile they like – in particular, there is nothing in the definition of a social choice function that requires me to report my preferences *truthfully*. But it should then be clear that participating in a social choice procedure has the character of a game: I have preferences over the possible outcomes Ω, and the actions or strategies that I have available to me are all the possible preference orders that I can declare. And, as we saw in Chapter 11, I will reason strategically about which action to perform, attempting to bring about the best possible outcome for myself that I can. And this action may not correspond to me reporting my preferences truthfully, as the various discussions above demonstrated. This raises an interesting question: *to what extent can we engineer a voting procedure that is immune to such manipulation?*

To make this issue precise, we first define what we mean by a voting procedure being susceptible to manipulation. Given a social choice function f, we say that f is *manipulable* if, for some collection of voter preference profiles $\omega_1, \ldots, \omega_i, \ldots, \omega_n$ and a voter i, there exists

some ω'_i such that

$$f(\omega_1, \ldots, \omega'_i, \ldots, \omega_n) \succ_i f(\omega_1, \ldots, \omega_i, \ldots, \omega_n).$$

Thus a voting procedure is manipulable in this sense if a voter can obtain a better outcome for themselves by unilaterally changing their preference profile, i.e. by misreporting their preferences to the voting procedure. Again, note that we have already seen voting procedures that are manipulable in this sense: recall the Labour supporter, above, who votes for the Liberal Democrats instead of Labour.

Broadly speaking, manipulability in this sense is not desirable, and so an obvious question is now whether we can engineer voting procedures that are *not* manipulable in this sense. Do such voting procedures exist? The answer is yes: and indeed, we have already seen an example: a dictatorship! You may be experiencing a sense of *déjà vu* at this point. So, let us phrase the question more carefully. Let's assume, as earlier, that there are at least three possible outcomes ($|\Omega| > 2$). Now, in this setting, do there exist any non-manipulable voting protocols, other than a dictatorship, which satisfy the Pareto condition? The Gibbard–Satterthwaite theorem tells us that the answer to this question is *no*.

Is it Bad to Tell Lies?

Our mothers taught us that it is bad to lie; but *why*, exactly, should we care about whether an agent lies in the context of a voting procedure? Why don't we just accept that everybody will act – strategically – in their best interests? If we know that everybody will do this, and we designed our procedure to give everybody equal power, then what's the problem?

> [In this context] freedom of speech means a right to lie, and taking into account [strategic manipulation] amounts to recognizing that the agent will [be sincere] only if it is selfishly rational to do so.
>
> [Moulin, 1983, p. 3]

On one level of analysis, of course, it is only rational for us to accept that everybody else will act rationally in the voting scenario, and so in this sense, and as long as everyone takes this into account, then there is no issue. But there are several reasons why we might favour procedures in which truthfully reporting preferences is rational. Suppose we have a voting procedure that has some desirable properties, but that (as Gibbard–Satterthwaite tells us) is prone to strategic manipulation. It may well be *computationally complex* for an agent to determine how to act in its best interest; acting rationally in this setting may perhaps not be computationally feasible at all. Apart from anything else, such a scenario favours agents with more processing power. It would be desirable in such a case to have a voting procedure in which truthfully reporting preferences was the rational course of action – then we would not have to expend any effort on doing anything except determining what our preferences are. Additionally, there is a result in game theory known as the *revelation principle* [Osborne and Rubinstein, 1994, p. 181], which (very) roughly says the following: suppose we have some procedure whose properties (i.e. equilibria) we like; then there exists another mechanism with the same properties except that truthfully reporting preferences is the rational course of action in this mechanism. Now, given the choice between a mechanism that has nice properties but is perhaps computationally expensive to manipulate, and one that has the same properties but in which telling the truth is rational, we would surely prefer the truth-telling

procedure, since it obviates the need for potentially expensive strategic reasoning.

Although it is perhaps less well known than Arrow's theorem, the Gibbard– Satterthwaite theorem seems similarly devastating in its implications for democracy.[1] But computer science – in the unlikely guise of computational complexity and NP-hardness – can come to the rescue.

The complexity of manipulation

The Gibbard–Satterthwaite theorem establishes that strategic manipulation is, to all intents and purposes, always possible. However, it does not tell us *how* such manipulation might be done, and it does not tell us that such manipulation is possible *in practice*. It simply tells us that manipulation is possible *in principle*.

COMPLEXITY OF MANIPULATION

So, how easy (or hard) is it to manipulate a voting procedure? This question was investigated in [Bartholdi et al., 1989] – and the answer turned out to be quite interesting. First, let us make a distinction between a voting procedure being *easy to compute* and *easy to manipulate*. By 'easy to compute', we simply mean that the function f can be implemented by an algorithm that runs in time polynomial in the number of voters and candidates. (The Slater ranking was the only example we saw that was hard to compute; the other voting procedures we looked at are easy to compute.) By 'easy to manipulate', we mean that if it is possible for a voter i to obtain a more preferred outcome by declaring a preference order ω'_i rather than its true preference order ω_i, then such a ω'_i can be computed in polynomial time. Now, in the same vein as above, let us ask the following question: Are there non-dictatorial voting procedures that are easy to compute and that satisfy the Pareto condition but that are *not* easy to manipulate? For once, the answer is good news: *yes*. In fact, a voting procedure called *second-order Copeland* satisfies these requirements. (In fact, this procedure also satisfies some other desirable properties, such as the Condorcet winner property. The details of second-order Copeland are not too important here; the point is that the procedure is hard to manipulate.)

SECOND-ORDER COPELAND

This result tells us that, while manipulation of second-order Copeland is possible *in principle*, to actually do such manipulation *in practice* may be too computationally complex. Thus computational complexity becomes a *positive* property, in complete contrast to the conventional situation in computer science, where we view complexity as a negative thing! A similar situation arises in cryptography, where the practical difficulty of finding factors for large numbers prevents decryption by an eavesdropper.

Although this is an exciting and tantalizing result, there is a catch. NP-completeness is a *worst case* result. It could be that, in fact, for all instances of practical interest, second-order Copeland might be easy to manipulate, i.e. that the 'hard' instances do not occur in practice. There might, for example, be heuristics for manipulation that work well on cases of practical interest. This is a reasonable point, and not much is really known at the time of writing about this issue.

Notes and Further Reading

[Taylor, 1995] is an admirably clear and readable introduction to social choice, with many practical examples, and is extremely detailed, with carefully presented proofs. If you are mathematically advanced, then you might find the presentation somewhat slow at times, but everybody else (and, let's face it, that is the majority of us) will find it a very pleasant read. At the other end of the technical spectrum [Arrow et al., 2002] is an extremely detailed reference on social choice theory, and to the best of my knowledge represents the state of the art at the time of writing. The only drawback of this otherwise excellent text is that some chapters are very terse, and you will probably need many other references to properly understand all the material.

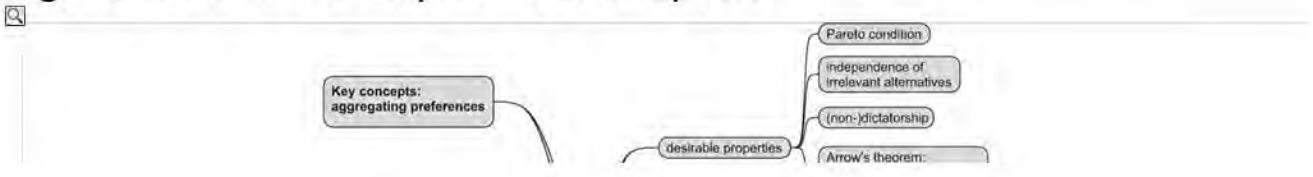
The Slater rule is one example of a voting procedure that is hard to compute, but there are others: [Hemaspaandra et al., 1997] analyse a voting procedure developed by the 18th-century Oxford academic Charles Dodgson (perhaps better known by his pen name Lewis Carroll), and show that finding a winner according to this procedure is complete for the complexity class 'parallel access to NP'. The details of this complexity class are perhaps unimportant here; the main point is that this is a negative result, intuitively worse than 'mere' NP-completeness.

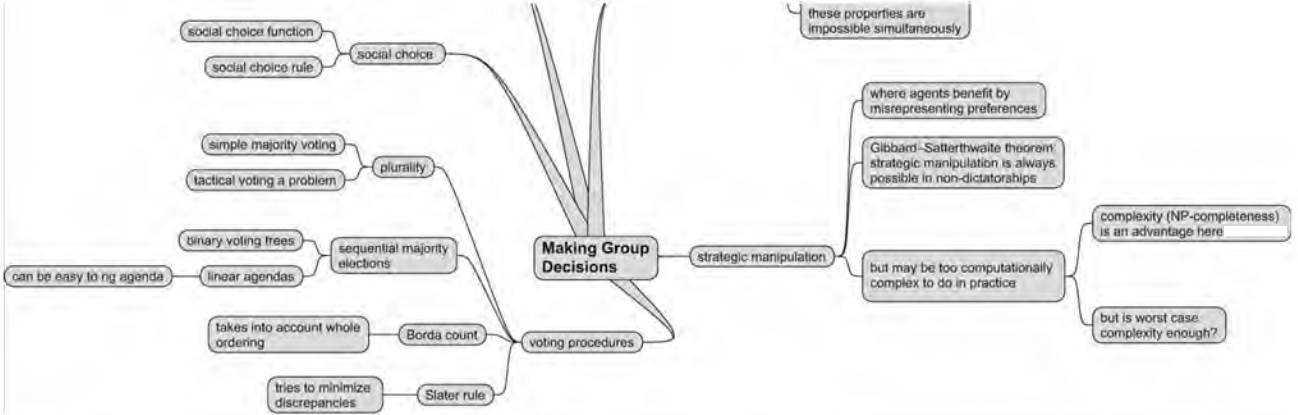
There are many, many other examples of voting procedures: [Brams and Fishburn, 2002] presents a comprehensive but rather technical survey, while [Taylor, 1995, 2005] provide less formal overviews. Arrow's theorem was proved in the 1950s [Arrow, 1951], and it was for this result that Kenneth Arrow was awarded the Nobel prize for economics in 1972. Arrow's theorem can be stated in many different ways, and the treatment used here follows the very instructive form in which the result is presented in [Taylor, 1995], although the actual version of the theorem is that stated in [Taylor, 2005, p. 19].

The fascinating results of [Bartholdi et al., 1989] spawned a raft of research aimed at establishing which voting protocols were computationally hard to manipulate, and under what circumstances. For example, implicit in the results of [Bartholdi et al., 1989] is an assumption that the number of candidates in the election is essentially unbounded. This raises the question of how many candidates are needed for manipulation to be hard [Conitzer et al., 2007]. On a slightly different tack, [Elkind and Lipmaa, 2005] discussed general approaches to designing hard-to-manipulate voting procedures, based on the idea of combining protocols. The average-case complexity of manipulating elections (as opposed to the worst-case complexity as characterized by NP-hardness results and similar) is considered in [Procaccia and Rosenschein, 2007]. Issues such as manipulation to try to prevent a particular outcome are studied in [Hemaspaandra et al., 2007]. Vincent Conitzer's thesis presents many results in this area [Conitzer, 2006a]. The COMSOC workshop series, founded in 2006, presents contemporary work in this area [Endriss and Lang, 2006].

Class reading: [Bartholdi et al., 1989]. This is the article that prompted the current interest in computational aspects of voting. It is a technical scientific article, but the main thrust of the article is perfectly understandable without a detailed technical background.

Figure 12.4: Mind map for this chapter.





[1]In fact, Arrow's theorem and the Gibbard–Satterthwaite theoremare equivalent: see [Taylor, 2005, pp. 69–72].

Chapter 13 Forming Coalitions

In the preceding chapters, we saw how ideas from economics, and in particular, concepts from game theory, could be used to analyse multiagent interactions. We also saw that using this analysis leads to some apparently troubling conclusions. We saw one scenario in particular – the prisoner's dilemma – where the game-theoretic analysis leads to an outcome that intuition suggests is suboptimal. We argued that cooperation cannot occur in the prisoner's dilemma because the conditions required for cooperation are not present. What were the features of this setting that prevented cooperation? We argue that the following features of the prisoner's dilemma prevented cooperation:

- binding agreements are not possible
- utility is given directly to individuals as a result of individual action.

The first feature means that agents cannot trust one another's promises: they must make their decisions based solely on the information they have about strategies and individual pay-offs, will try to maximize their own utility, and will assume that all other agents will be trying to do the same thing. Notice that, if binding agreements were possible in the prisoner's dilemma (i.e. the prisoners were able to get together beforehand and make a binding commitment to mutual cooperation), then there would indeed be no dilemma at all.

The second feature says that an agent need not be concerned with collective utility; it need only try to maximize its own utility and assume that everybody else will do likewise.

In many real-world situations, these assumptions simply do not hold. For example, we can often use contracts and other legal arrangements to make binding agreements with others. Additionally, the revenue that a company earns is not credited to an individual, but to the company itself. In such situations, cooperation may become both possible and rational. In short, this chapter looks at what happens when we drop these assumptions, which takes us into the realm of *cooperative game theory*.

COOPERATIVE GAMES

13.1 Cooperative Games

We first fix some terminology. We will deal with situations in which there are n agents (typically $n > 2$), and as usual we let $Ag = \{1, \ldots, n\}$ be the set of all these agents. We will refer to a subset of Ag as a *coalition*, and we use C, C', C_1, … to denote coalitions. The use of the term 'coalition' occasionally causes some confusion, since in everyday speech, when we say a group of people have formed a coalition, this implies some commitment to collective action. This is not what we mean here. A coalition in our sense is simply a set of agents, who may or may not work together. When we say the *grand coalition*, we mean the coalition consisting of all agents, $C = Ag$. Finally, a *singleton* coalition is a coalition containing just one agent. Now, recall that, above, we mentioned that we abstract away from the individual actions that agents can perform, and instead think of collective actions. In fact, we view collective actions at a very abstract level: we simply think of a coalition as having the ability to obtain a certain utility, which can be shared among coalition members.

COALITION

GRAND COALITION

Formally, we model a *cooperative game* (or *coalitional game*) as a pair

$$G = (Ag, v),$$

where Ag is a set of agents and

$$v: \mathbf{2}^{Ag} \to \mathbb{R}$$

is called the *characteristic function* of the game. The idea of the characteristic function is that it assigns to every possible coalition a numeric value, intuitively representing pay-off that may be distributed between the members of that coalition. That is, if $v(C) = k$, then this means that coalition C can cooperate in such a way that they will obtain utility k, which may then be distributed among team members. Before proceeding, there are several points to note about this definition:

CHARACTERISTIC FUNCTION

- The game itself is completely silent about how this utility should be distributed among coalition members: coalition members have to agree among themselves how to divide the 'pay cheque'.
- The origin of a characteristic function for any given scenario is generally regarded as not being a concern in cooperative game theory. It is assumed to simply somehow be 'given': $v(C)$ is simply the largest value that C could obtain by cooperating, and we are not concerned with *how* they cooperate at this level of modelling.

A *simple* coalitional game is one where the value of any coalition is either 0 (in which case we say the coalition is 'losing') or 1 (the coalition is 'winning'). Later on, we will see that some *voting systems* – notably, *weighted voting games* – can usefully be understood as simple games.

SIMPLE GAMES

Given a game of this type, cooperative game theory attempts to answer such questions as which coalitions might be formed by rational agents, and how the pay-off received by a coalition might be 'reasonably' divided between the members of a coalition. Just as noncooperative game theory has solution concepts (such as Nash equilibrium), so cooperative game theory has its own solution concepts.

Three stages of cooperative action

Sandholm et al. identify three key issues that have been addressed with respect to cooperative games [Sandholm et al., 1999, pp. 210–211], which we refer to as the *cooperation lifecycle* – see Figure 13.1.

> **Coalition structure generation** The first step in cooperative action involves the participants deciding who they will work with. How does an agent make this decision? What information does it have available? The standard assumption is that the only information available to participants is the characteristic function of the game: they know how much every coalition can earn. Now, an agent will seek to maximize its utility: intuitively, it wants to join a coalition that earns a lot. But every agent will reason likewise, and this is where strategic considerations arise in cooperative games

reason likewise, and this is where strategic considerations arise in cooperative games. Suppose that you are a hard-working agent, while I am lazy. I may be keen to join a coalition with you, since I know that you work hard, and we will earn much more than I would be able to earn on my own. But this isn't enough for us to make a successful coalition; you may be reluctant to join me, because you know that I am lazy. So here, the coalition containing you and me is said to be *unstable*, since one of us (you, the hard-working one) has an incentive to *defect* and join a coalition containing more productive agents. So, asking 'which coalitions will form?' amounts to asking 'which coalitions are stable?'. We make this idea precise with the notion of the *core*. We look at the core in Section 13.1.1.

COALITION STRUCTURE GENERATION

STABILITY

If a system is owned by a central designer, then the issue of how individual agents fare is not necessarily so important. We then typically aim to partition agents into coalitions so that the overall utility of the system (i.e. the social welfare) is maximized. This is *coalition structure generation*. We look at coalition structure generation in Section 13.6.

COALITION STRUCTURE GENERATION

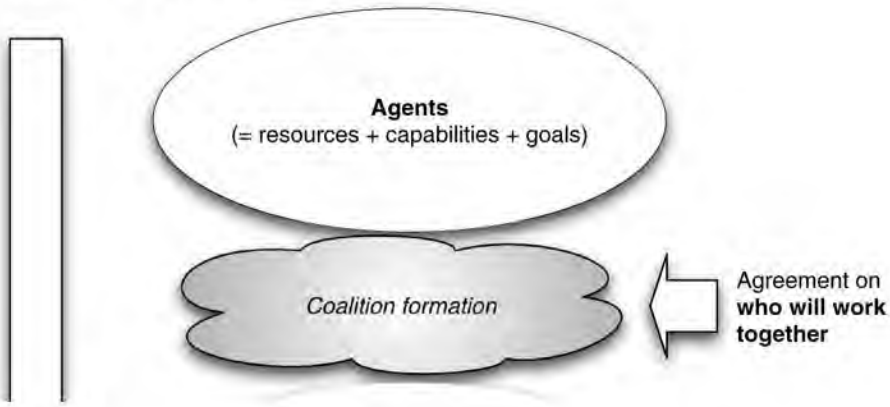
Solving the optimization problem of each coalition This is solving the 'joint problem' of a coalition, i.e. finding the best way to maximize the utility of the coalition itself. This implies that the coalition must decide upon collective plans and then carry out these plans to achieve the highest collective benefit.

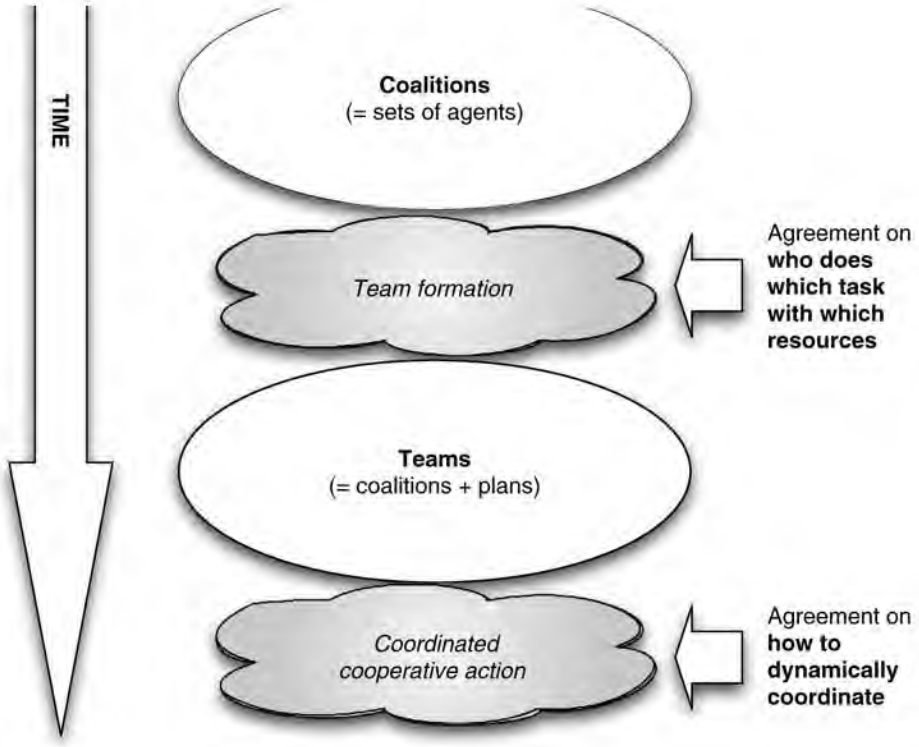
Dividing the value of the solution for each coalition This involves deciding 'who gets what' in the final pay-off of the game. A key issue here is that of *fairness*. With respect to this issue, concepts such as the Shapley value [Osborne and Rubinstein, 1994, p. 89] have been developed that attempt to answer the question of how much an agent should receive based on an analysis of how much that agent contributes to a coalition. We look at the Shapley value in Section 13.1.2.

FAIR DIVISION

In this chapter, we focus on the first and third issues; the second issue (how to effectively cooperate and coordinate cooperative actions) was discussed in Chapter 8.

Figure 13.1: The cooperation lifecycle.





13.1.1 The core

Imagine the following scenario. It is 9 a.m. on a working day, and you want to earn some money. You have certain skills and abilities, and you know that you can earn a certain amount of money on your own. But you also know that there are others you could work with, and that it may be possible, by cooperating, to generate some *surplus* income, over and above the amount that could be earned by individuals. That is, there may be *added value* to cooperation. So, who will you choose to work with? Intuitively, the answer is, of course, that you want to work with a coalition that earns as much as possible. Practically, this might mean choosing to work with other agents that are highly skilled, hard working, and so on, with complementary skills to your own. But, of course, we have to assume that all the agents in the system reason in the same way – they are also trying to join a coalition that is as productive as possible. This implies that we cannot simply join any coalition we choose. A coalition can only form if *all* the agents in it *prefer* to be in it. In particular, a coalition will only form if *none* of the participants can do better by *defecting* and forming a coalition on their own. In this way, we reduce the question *Which coalition should I join?* to the question

Which coalitions are stable?

Notice that stability is a *necessary* but not *sufficient* condition for coalitions to form. That is, an unstable coalition will never form; however, the fact that a coalition *is* stable does not guarantee that it *will* form (for example, there could be multiple stable coalitions). In what follows, we consider the stability of the grand coalition, as this makes the analysis a little easier. Thus, we will be asking whether the grand coalition is stable.

We make the idea of coalitional stability concrete in the notion of the *core*. Roughly, the *core* of a coalitional game is the set of *feasible* distributions of pay-off to members of a coalition that *no* subcoalition can reasonably object to. Let us make this idea precise. An *outcome* $\mathbf{x}$ for a coalition C in a game (Ag, v) is a distribution of C's utility to members of C, $\mathbf{x} = (x_1, \ldots, x_k)$, which is both feasible for C (in the sense that C really could obtain the pay-off indicated in $\mathbf{x}$) and *efficient* (in the sense that all available utility is allocated). Formally, an outcome $\mathbf{x} = (x_1, \ldots, x_k)$ for coalition $C = \{1, \ldots, k\}$ must satisfy the following property:

$\ldots, x_k)$ for coalition $C = \{1, \ldots, k\}$ must satisfy the following property:

THE CORE

$$\sum_{i \varepsilon c} x_i = v(c).$$

For example, let's suppose we have a game with $Ag = \{1, 2\}$, such that $v(\{1\}) = 5$, $v(\{2\}) = 5$, and $v(\{1, 2\}) = 20$. Then the possible outcomes are (20, 0), (19,1), (18,2), …, (0,20). The outcome (20, 0) says that agent 1 gets the entire collective value, while agent 2 gets nothing; in the outcome (19, 1), agent 1 gets 19 while agent 2 gets 1; and so on. Actually, of course, there will be infinitely many possible outcomes, since we assume the pay-off can be divided as finely as we like between the agents – but you get the idea.

Now, we will say that a coalition C *objects* to an outcome for the grand coalition if there is some outcome for C that makes *all members of C strictly better off*. Formally, $C \subseteq Ag$ objects to an outcome $(x_1, \ldots, x_n)$ for the grand coalition if there is some outcome $(x'_1, \ldots, x'_k)$ for C such that

$$x'_i > x_i \text{ for all } i \in C.$$

Now, clearly, an outcome is not going to happen if some coalition objects to it – they would do better by defecting and working on their own. So think about the outcomes for the grand coalition to which nobody has any objection. We say that the *core* is the set of outcomes for the grand coalition to which no coalition objects. If the core is *non-empty* then there is some way that the grand coalition can cooperate and distribute the resulting utility among themselves such that no coalition could do better by defecting. Thus the question

Is the grand coalition stable?

is the same as asking:

Is the core non-empty?

Let us return to the example above. Is the core non-empty? The obvious way to think about this is to systematically enumerate all outcomes for the grand coalition, and ask whether any coalition objects. Starting with (20, 0), agent 2 clearly objects, since it can do better on its own: $v(\{2\}) = 5$. Similarly, as we progress through (19,1), (18, 2), agent 2 continues to object, until we reach (15, 5). Now, at this point, 2 ceases to have any objection in the sense that we described above, since it cannot actually do better on its own. Of course, in an informal sense 2 might object to (15, 5), since all the surplus generated by cooperation goes to 1. Agent 2 would probably argue that outcome (15,5) is *unfair*, but the point of the core is not to study fairness but stability. We will consider fairness in the following section. As we proceed through (14, 6), we continue to have no objections; then, once we pass (5,15), for example considering (4,16), agent 1 starts to object all the way to (0, 20). Thus the core of this game is non-empty, and contains all the outcomes between (15,5) and (5,15) inclusive.

The core is an intuitive and natural formulation of coalitional stability, and is probably the most-studied concept in cooperative games. However, there are some problems with the core as a solution concept for coalitional games, chief among them being the following:

- Sometimes, the core is empty; what happens then?
- Sometimes the core is non-empty but isn't 'fair', as the above example illustrates.

- The definition of the core involves quantification over all possible coalitions (since it requires that no subset of agents has any incentive to defect and form a coalition on their own). Now, if the system contains n agents, there will be $2^n - 1$ subsets, and this suggests that checking coalitional stability will be hard; we will see below that this is indeed often the case, for realistic representations of coalitional games.

The core seems most useful as a mechanism for evaluating the stability of coalitions. However, while an acceptable outcome will probably be in the core, there is nothing in the definition of the core itself to indicate *which* outcome should be implemented. For this reason, we now introduce the Shapley value.

13.1.2 The Shapley value

Recall that, in the example above, outcomes such as (15,5) and (5,15) were regarded as being intuitively *unfair*. So, what might be a fair solution? Intuition suggests that, for this scenario, a 50:50 split of income is reasonable. In general, though, simply dividing income evenly does not seem like a reasonable solution. Apart from anything else, simply dividing surplus evenly without taking into account performance gives no incentive for agents to perform well. A more meritocratic approach would be to *divide surplus according to contribution:* the better an agent performs, relative to its peers, the higher its reward. The *Shapley value* is a solution for coalitional games which makes this idea precise, and, as we shall see, it has some extremely elegant properties.

SHAPLEY VALUE

Lloyd Shapley, the inventor of the Shapley value, started by defining a number of axioms that, he argued, any fair distribution of coalitional value should satisfy. These three axioms are known as *symmetry, dummy player*, and *additivity*. We will start by formally defining these axioms, for which we need some more notation. Let $\mu_i(C)$ be the amount that i adds by joining $C \subseteq Ag \backslash \{i\}$ – this is the *marginal contribution of i to C*:

MARGINAL CONTRIBUTION

$$\mu_i(C) = v(C \cup \{i\}) - v(C).$$

Note that if $\mu_i(C) = v(\{i\})$, then there is no 'synergy' or 'added value' from i joining C, since the amount i adds is exactly what i would earn on its own. Finally, sh_i will denote the value that agent $i \in Ag$ is given in the game (Ag, v).

Symmetry The *symmetry* axiom says that *agents that make the same contribution should get the same pay-off*. Another way to think about this axiom is as saying that the amount an agent gets should *only* depend on their contribution, and not on their name (or race, religion, sexual orientation, etc.). Then i and j are said to be *interchangeable* if $\mu_i(C) = \mu_j(C)$ for every $C \subseteq Ag \setminus \{i, j\}$. The symmetry axiom then says that *if i and j are interchangeable then* $sh_i = sh_j$.

SYMMETRY AXIOM

Dummy player The *dummy player* axiom says that agents that never have any synergy

with any coalition should get only what they can earn on their own. Formally, let us say $i \in Ag$ is a *dummy player* if $\mu_i(C) = v(\{i\})$ for every $C \subseteq Ag \setminus \{i\}$. That is, a dummy is a player that only ever adds to a coalition the value that it could obtain on its own. The dummy player axiom then says that *if i is a dummy player then* $sh_i = v(\{i\})$.

DUMMY PLAYER AXIOM

Additivity This assumption is a little technical, and less straightforwardly intuitive than the other two. It basically says that if you combine two games, then the value that a player gets should be the sum of the values it gets in the individual games: a player does not gain or lose by playing more than once. More formally, let $G^1 = (Ag, v^1)$ and $G^2 = (Ag, v^2)$ be games containing the same set of agents, let $i \in Ag$ be one of the agents, let sh_i^1 and sh_i^2 be the value player i receives in games G^1 and G^2 respectively, and let $G^{1+2} = (Ag, v^{1+2})$ be the game such that $v^{1+2}(C) = v^1(C) + v^2(C)$. Then the additivity property says that the value sh_i^{1+2} of player i in game G^{1+2} should be $sh_i^1 + sh_i^2$.

ADDITIVITY AXIOM

We now have some properties that intuitively define what constitutes a fair distribution of coalitional value. But we have not seen how to *compute* such a value. Shapley's idea was that an agent should get *the average marginal contribution it makes to a coalition*. This suggests that, as a first attempt, we might find a value for agent i by summing up, over all possible coalitions C not containing i, the marginal contribution $\mu_i(C)$ that i makes to C, and then dividing by the total number of such coalitions, i.e.

AVERAGE MARGINAL CONTRIBUTION

$$\frac{1}{2^n - 1} \sum_{C \subseteq Ag \setminus \{i\}} \mu_i(C) \qquad (13.1)$$

In fact, Equation (13.1) is not quite right, although it does serve a useful purpose in *weighted voting games*, where it is called the *Banzhaf index;* we will see weighted voting games below. Why is (13.1) not right? Because the way it measures a player's marginal contribution to a coalition C takes no account of the *order* in which the coalition C is formed. Intuitively, we can imagine that if i joins the coalition when it has few members, it is likely to be adding quite large value to the coalition, while if it joins the coalition as the last member, the difference it makes is likely to be much smaller. To properly consider the marginal contribution that an agent makes to a coalition, therefore, we must consider the average value it adds *over all possible positions that it could enter the coalition*. We will now define this value formally, for which we need a few more definitions. Let $\Pi(Ag)$ denote the set of all possible orderings of the agents Ag; so, for example, if $Ag = \{1,2,3\}$ then

BANZHAF INDEX

$$\Pi(Ag) = \{(1, 2, 3), (1, 3, 2), (2, 1, 3), (2, 3, 1), (3, 1, 2), (3, 2, 1)\}.$$

If $o \in \Pi(Ag)$, then let $C_i(o)$ denote the agents that appear before i in the ordering o (so if $o = (3, 1, 2)$, then $C_3(o) = \emptyset$, $C_1(o) = \{3\}$, and $C_2(o) = \{1, 3\}$).

Then the Shapley value for agent i, denoted sh_i, is defined as:

$$sh_i = \frac{1}{|Ag|!} \sum_{o \in \Pi(Ag)} \mu_i(C_i(o)). \tag{13.2}$$

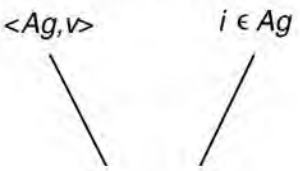
So, how fair is the Shapley value? Well, the first thing to say is that it satisfies all three of the fairness axioms above. However, Shapley proved something much stronger than this: he proved that (13.2) is the *unique* value that satisfies the fairness axioms. Why is this important? Well, suppose you think that Shapley's fairness axioms are not enough, and you have your own notion of fairness that you want to define. You then construct some definition, in the spirit of Equation (13.2), to give this value. Then Shapley's result says that *if the value you define satisfies the fairness axioms given above, then your value* is *the Shapley value as defined in Equation (13.2)!* Of course, there are various different ways of *presenting* the Shapley value, and Shapley's result has nothing to say about those. The point is that no matter what your equation looks like, if it satisfies the fairness axioms, then it gives the Shapley value as defined in (13.2).

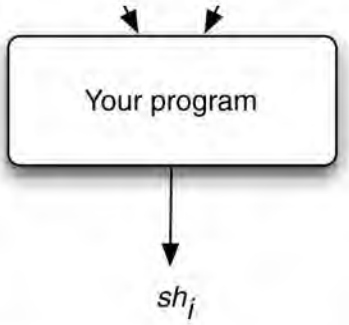
Let's consider some examples.

- Consider the game introduced above, in which $v(\{1\}) = 5$, $v(\{2\}) = 5$, and $v(\{1, 2\}) = 20$. Intuition says that an allocation of 10 to each agent would be fair for this game. We have $\mu_1(\emptyset) = 5$, $\mu_1(\{2\}) = 15$, $\mu_2(\emptyset) = 5$, $\mu_2(\{1\}) = 15$, and so $sh_1 = sh_2 = (5 + 15)/2 = 10$. Our intuition was thus correct: both players should get the same amount.
- Suppose we have $Ag = \{1, 2\}$, with $v(\{1\}) = 5$, $v(\{2\}) = 10$, and $v(\{1, 2\}) = 20$. Here, we have $\mu_1(\emptyset) = 5$, $\mu_1(\{2\}) = 10$, $\mu_2(\emptyset) = 10$, $\mu_2(\{1\}) = 15$, and so $sh_1 = (5 + 10)/2 = 7.5$ and $sh_2 = (10 + 15)/2 = 12.5$.
- Finally, suppose we have $Ag = \{1, 2, 3\}$, with $v(\{1\}) = 5$, $v(\{2\}) = 5$, $v(\{3\}) = 5$, $v(\{1, 2\}) = 10$, $v(\{1, 3\}) = 10$, $v(\{2, 3\}) = 20$, and $v(\{1,2,3\}) = 25$. We have $\mu_1(\emptyset) = 5$, $\mu_1(\{2\}) = 5$, $\mu_1(\{3\}) = 5$, $\mu_1(\{2,3\}) = 5$, $\mu_2(\emptyset) = 5$, $\mu_2(\{1\}) = 5$, $\mu_2(\{3\}) = 15$, $\mu_2(\{1,3\}) = 15$, $\mu_3(\emptyset) = 5$, $\mu_3(\{2\}) = 15$, and $\mu_3(\{1,2\}) = 15$. It should be immediately clear that agent 1 is a dummy player, and so we can conclude, without any further arithmetic, that $sh_1 = 5$ (recall the dummy player axiom). For agent 2 we have $sh_2 = (5 + 5 + 15 + 15)/4 = 10$ and by the same arithmetic, $sh_2 = 10$.

This last example illustrates that the naive approach to computing the Shapley value requires time exponential in the number of players; this is because we need to consider each possible permutation of the agents. This observation seems like an appropriate point at which to start considering specifically computational aspects of coalitional games.

Figure 13.2: Computing with coalitional game models.





13.2 Computational and Representational Issues

Impressed and intrigued by the ideas set out so far in this chapter, you decide to write a program to implement some of them. You want a program along the lines shown in Figure 13.2: you want a program that will take as input a coalitional game (Ag, v) and a player $i \in Ag$, and produces as output the Shapley value sh_i. But as you begin to think about the design of the program, an interesting question arises: how do you *represent* the game in the input to your program? As a first attempt, you might choose to define the characteristic function as a text file looking something like this (representing the last example we gave above):

```
1,2,3
1 = 5
2 = 5
3 = 5
1,2 = 10
1,3 = 10
2,3 = 20
1,2,3 = 25
```

The first line defines the set of players; subsequent lines define the value of every coalition. Well, such a representation would clearly work in principle; but it would be *utterly infeasible* in practice, for anything other than a tiny number of players. Why? Because an n-player game would require an input file with $2^n + 1$ lines; a 100-player game, for example, would require a text file with 1.2×10^{30} lines.

So, what we typically want is a way of *succinctly* or *concisely* representing a coalitional game, and crucially, the characteristic function of a coalitional game. In particular, what we want is some way of representing a characteristic function so that our input will be of size *polynomial* in the number of agents. However, as a general rule, the more succinct or concise any given representation scheme is, the harder it is to compute with. We therefore typically seek a representation that strikes a useful balance between *succinctness* and *tractability*.

SUCCINCT REPRESENTATIONS

13.3 Modular Representations

The first representations we will look at are attractive because they directly exploit Shapley's axioms when providing a route to computational efficiency. We call these *modular* representations, because they are based on the idea of dividing an overall game down into a number of smaller games; as we will see, we can then directly appeal to Shapley's additivity

axiom to compute the Shapley value.

MODULAR REPRESENTATIONS

13.3.1 Induced subgraphs

The first representation we look at was introduced by [Deng and Papadimitriou, 1994]. The idea of the representation is very simple. A characteristic function is defined by an undirected, weighted graph, in which nodes in the graph are the members of Ag. We assume that edges are just labelled with integer values for now. We let $w_{i,j}$ be the weight of the edge from i to j in the graph. Then, to compute the value of a coalition $C \subseteq Ag$ we simply sum up over all the edges in the graph whose components are all contained in C – so the value of a coalition $C \subseteq Ag$ is the weight of the *subgraph induced by* C:

$$v(C) = \sum_{\{i,j\} \subseteq C} w_{i,j}.$$

Figure 13.3 illustrates the idea. In Figure 13.3a, we see the basic graph defining a characteristic function for four agents, $\{A, B, C, D\}$. Figure 13.3b shows the subgraph induced by the coalition $\{A, B, C\}$, while Figure 13.3c shows the subgraph induced by the singleton coalition $\{D\}$, and finally, Figure 13.3d shows the subgraph induced by $\{B, D\}$. From this, we can see that $v(\{A, B, C\}) = 3 + 2 = 5$, while $v(\{D\}) = 5$, and $v(\{B, D\}) = 5 + 1 = 6$.

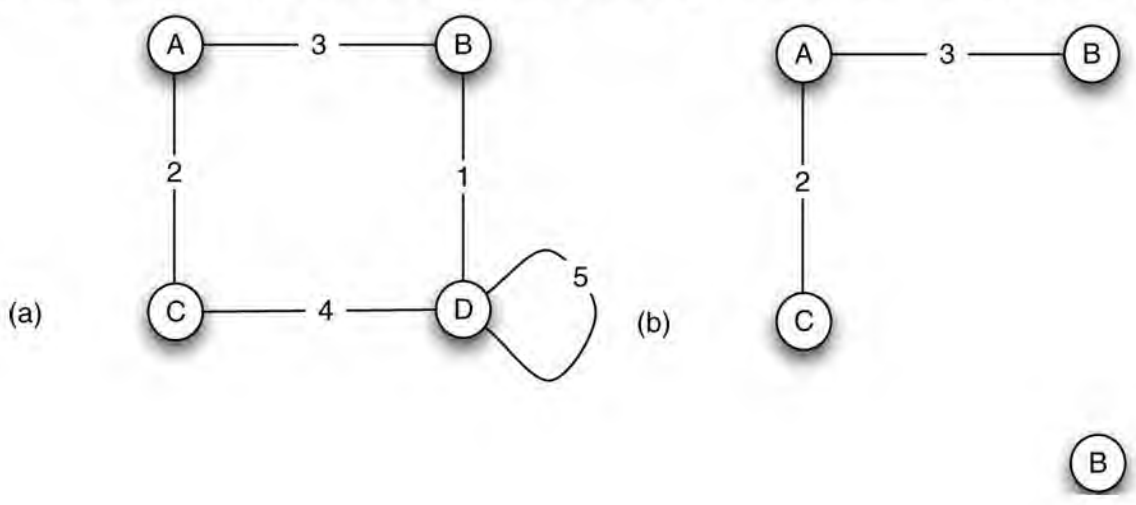
Now, it should be easy to see that this representation is succinct: using an adjacency matrix to represent the graph will require only $|Ag|^2$ entries, for example. Of course, this is not a *complete* representation: there are characteristic functions v that cannot be represented using such a graph. For example, consider a game $G_{silly} = (Ag, v_{silly})$ in which

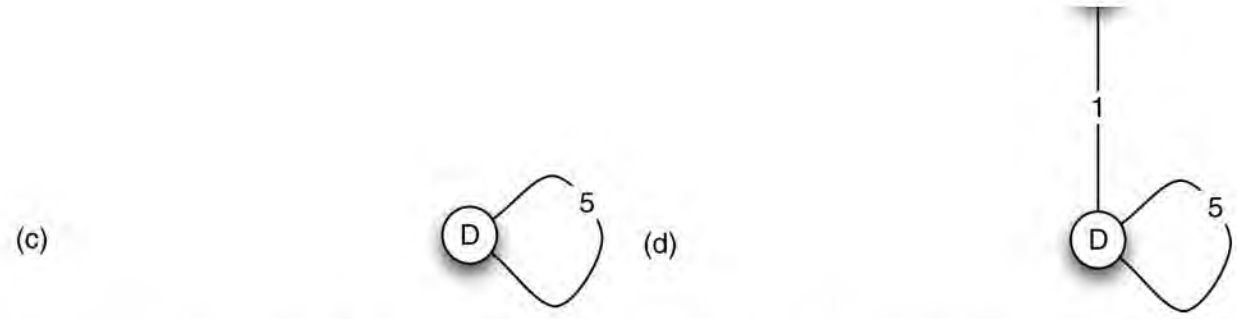
COMPLETE REPRESENTATIONS

$$v_{silly}(C) = \begin{cases} 1 & \text{if } |C| > 1 \\ 0 & \text{otherwise.} \end{cases}$$

This is, admittedly, perhaps not a very sensible coalitional game, but it *is* a coalitional game, and it turns out that we cannot represent it using the induced subgraph representation. However, incompleteness is not an issue if a representation can capture the scenarios we are interested in.

Figure 13.3: Representing coalitional games with a weighted graph.





So, we have a succinct – but incomplete – representation. How hard is it to compute with this representation? The first thing to note is that, despite its rather fearsome combinatorial appearance, it is possible to compute the Shapley value for a given player in a game represented using the induced subgraph representation in polynomial time. The proof is sufficiently simple, and appealing, that it is worth hearing about, not least because it provides the key to other, similar modular representations for coalitional games.

Consider the characteristic function graph in Figure 13.3a. Now, we can think of this graph as really being the combination of five different games, with one game for each edge: for example, there is a game $A \overset{3}{-} B$, a game $A \overset{2}{-} C$, and so on. Notice that each of these component games has exactly two players. The next step is to realize that, by Shapley's additivity axiom, we can compute the Shapley value of a player in the overall game by simply adding together the Shapley values for that player in each of the component games. So, if we knew the Shapley value for the player in each of the component games, we could easily compute its overall Shapley value. Now, consider one of the individual games, for example $A \overset{3}{-} B$. What is agent A's Shapley value from this game? The point here is to notice that the agents are *interchangeable*, and therefore, by Shapley's symmetry axiom, they should each get the same amount from this game. Since there are two players in the game, it follows that they should each get half of the income, 3, from the game, so each player in this game has a Shapley value of 1.5. We can apply the same reasoning in each of the component games: in each component game, a player gets half the value of the weight on the edge. Overall, therefore, an agent gets *half the income from the edges in the graph to which it is attached*, that is:

$$sh_i = \frac{1}{2}\sum_{j \neq i} w_{i,j}.$$

Turning to other properties that we might want to check, checking emptiness of the core is NP-complete, while checking whether a specific outcome is in the core is co-NP-complete.

13.3.2 Marginal contribution nets

The *marginal contribution nets* representation was introduced in [Ieong and Shoham, 2005], and can be understood as an extension to the induced subgraph representation discussed above. The basic idea behind marginal contribution nets is to represent the characteristic function of a game as a set of rules, of the form:

MARGINAL CONTRIBUTION NET

pattern → value.

Here, the pattern is simply conjunction of agents; a rule *applies* to a group of agents C if C is a superset of the agents in the pattern. For example, a rule with pattern $1 \wedge 3$ would apply to

the coalitions {1, 3}, {1, 3, 5}, but not to the coalitions {1} or {8, 12}. Where $\varphi \rightarrow x$ is a rule and C is a coalition, we write $C \vDash \varphi$ to mean that the rule applies to coalition C. Given a set of rules rs, we write rs_C to mean the subset of rs that apply to coalition C, that is:

$$rs_C = \{\varphi \rightarrow x \in rs \mid C \vDash \varphi\}.$$

Given a set of rules rs, we compute the value of a given coalition C by summing over the values of all the rules that apply to the coalition. Expressed a bit more formally, the characteristic function v_{rs} associated with ruleset rs is defined as follows:

$$v_{rs}(C) = \sum_{\phi \rightarrow x \in rs_C} x.$$

Let us consider an example. Suppose we have ruleset rs_1, containing the following rules:

$$a \wedge b \rightarrow 5$$

$$b \rightarrow 2.$$

We have: $v_{rs1}(\{a\}) = 0$, $v_{rs1}(\{b\}) = 2$, and $v_{rs1}(\{a, b\}) = 7$.

Notice that the condition part of a rule is a logical formula: an obvious extension is to allow negations in rules (indicating the absence of agents, rather than their presence). For example, consider rs_2:

$$a \wedge b \rightarrow 5$$

$$b \rightarrow 2$$

$$c \rightarrow 4$$

$$b \wedge \neg c \rightarrow -2.$$

Here, we have $v_{rs2}(\{a\}) = 0$ (because no rules apply), $v_{rs2}(\{b\}) = 0$ (second and fourth rules), $v_{rs2}(\{c\}) = 4$ (third rule), $v_{rs2}(\{a, b\}) = 5$ (first, second, and fourth rules), $v_{rs2}(\{a, c\}) = 4$ (third rule), and $v_{rs2}(\{b, c\}) = 6$ (second and third rules).

Now, using the same idea as we used for the induced subset representation, we can show that it is possible to compute the Shapley value for a given agent for a given marginal contribution net in polynomial time. Consider the first rule: $a \wedge b \rightarrow 5$. This rule conveys exactly the same information as a link from a to b weighted with 5 in the induced subgraph representation. That is, because the Shapley value obeys the additivity property, we can treat every different rule as a different game, work out the Shapley value for each rule, and then simply sum over all the rules to get the overall Shapley value. Computing the Shapley value for a player in the game $a \wedge b \rightarrow 5$ is easy because we can apply Shapley's symmetry axiom: there is no way to distinguish the contributions that the players make, so they must both get the same Shapley value. Thus sh_a for the rule $a \wedge b \rightarrow 5$ is clearly 2.5. So, consider a rule set rs in which conditions are of the form $1 \wedge \ldots \wedge l$, then it should be clear that we get

$$sh_i = \sum_{r \in rs \,:\, i \text{occurs in lhs of} r} sh_i^r$$

where

$$sh_i^{1 \wedge \ldots \wedge l \rightarrow x} = \frac{x}{l}.$$

Things get a little more complicated when we have negations in rule conditions (e.g. we have conditions such as $1 \wedge 3 \wedge \neg 7$), but the same basic idea can be used [Ieong and Shoham, 2005].

Of course, since marginal contribution nets are a generalization of the induced subgraph representation, we cannot expect problems involving the core to be any easier than they are with the induced subgraph representation. However, they are, at least, no harder.

Unlike the induced subgraph representation, however, marginal contribution nets are a *complete* representation: any characteristic function can be captured by a marginal contribution nets representation (we need negations in rules to do this, in which case we can in the worst case use one rule for every possible coalition C). In the worst case, however, we may need exponentially many rules in the number of agents in the game.

13.4 Representations for Simple Games

So far, we have focused on representations for coalitional games in general – where the value of a coalition may be any value. However, certain restricted types of coalitional games have simple and natural representations. One such class of coalitional games are the so-called *simple* games: recall that a simple coalitional game is one in which every coalition gets a value of either 1 ('winning') or 0 ('losing'). Simple games are important because they model *yes/no* voting systems: voting systems in which a proposal (e.g. a new law, or a change to tax regulations) is pitted against the status quo [Taylor and Zwicker, 1999]. Decision-making in most political bodies can be understood as a yes/no voting system (e.g. in the United Kingdom, the voting system of the House of Commons; in the European Union, the voting system in the enlarged EU; in the USA, the US Federal System; in the United Nations, the voting system of the Security Council [Taylor, 1995]).

YES/NO GAME

One way to model a yes/no voting game is as a pair $Y = (Ag, W)$, where $Ag = \{1, \ldots, n\}$ is the set of agents, or voters, and $W \subseteq \mathbf{2}^{Ag}$ is the set of *winning coalitions*, with the intended interpretation that, if $C \in W$, then C would be able to determine the outcome (either 'yes' or 'no') to the question at hand, should they collectively choose to. Several possible conditions on yes/no voting games suggest themselves as being appropriate for some (though of course not all) scenarios:

- *Non-triviality* There are some winning coalitions, but not all coalitions are winning – formally, $\emptyset \subset W \subset \mathbf{2}^{Ag}$.
- *Monotonicity* If C wins, then every superset of C also wins – formally, if $C_1 \subseteq C_2$ and $C_1 \in W$ then $C_2 \in W$.
- *Zero-sum* If a coalition C wins, then the agents outside C do not win – formally, if $C \in W$ then $Ag \setminus C \notin W$.
- *Empty coalition loses:* $\emptyset \notin W$.
- *Grand coalition wins:* $Ag \notin W$.

(Of course the final two conditions together imply the first, but not vice versa.)

Now, as with coalitional games in general, the naive representation of (Ag, W) (explicitly

listing all of the winning coalitions) will be exponential in the number of agents. We therefore need more concise representations. One such representation is the *weighted voting game*.

13.4.1 Weighted voting games

The idea of a weighted voting game is simple and natural: for each agent $i \in Ag$ we define a weight w_i, and we define an overall *quota*, q. A coalition C is then winning if the sum of their weights exceeds the quota:

WEIGHTED VOTING GAME

$$v(C) = \begin{cases} 1 & \text{if} \sum_{i \in C} w_i \geq q \\ 0 & \text{otherwise}. \end{cases}$$

A weighted voting game with players 1, …, n, having weights w_1, …, w_n and quota q is written as $(q; w_1, \ldots, w_n)$.

Weighted voting games are of course very widely used in the real world, and *simple majority voting* is a special case. Simple majority voting is the electoral process used in the UK and many other countries. Players are the people voting, and each player has weight $w_i = 1$; for the quota, we have

$$q = \frac{\lceil |Ag| + 1 \rceil}{2}.$$

Where they are appropriate, weighted voting games are succinct, since we only need to represent the weight of each player and the quota. So, how hard is it to compute the solution concepts discussed above? The news is not good for the Shapley value: it is NP-hard to compute the Shapley value for weighted voting games [Deng and Papadimitriou, 1994], and moreover, it can be proved that there is no polynomial time algorithm that will be guaranteed to give a reasonable approximation of this value. This is disappointing, as the Shapley value in weighted voting games has an interesting interpretation: it measures the *power* of a voter – the ability of a voter to influence the political decision-making process. In fact, the Shapley value has a special name when interpreted in yes/no voting systems: it is called the *Shapley–Shubik power index* [Felsenthal and Machover, 1998]. One might ask why such a value is necessary, for surely the weight of a voter directly indicates their power – the larger the weight, the more powerful the voter. This is certainly true in a sense, but it is in fact a little misleading. Consider the following weighted voting game, with agents $Ag = \{1, 2, 3\}$:

SHAPLEY–SHUBIK INDEX

$$(100; 99{,}99, 1).$$

Intuition suggests that since agents 1 and 2 have weights nearly 100 times larger than that of agent 3, then they must be more powerful, but of course this is not the case. *Any coalition containing at least two players is winning: the three agents are interchangeable, and so have equal Shapley values* $\left(\frac{1}{3}\right)$. This phenomenon is of course well known in the real world, where small political parties can wield great power when elections have marginal results. This happened in the UK, for example, between 1992 and 1997, when the ruling Conservative

Party was forced to make concessions to small parties in order to ensure that they were able to pass laws through Parliament.

Sometimes, the fact that a voter has a non-zero power is meaningless. For example, consider the following weighted voting game:

$$(10; 6, 4, 2).$$

Here, the final player is *never* able to influence a decision – the only winning coalitions are those containing at least the first two players, and adding the final player to a coalition never changes the status of a coalition. The final player here has a Shapley value of 0 (sometimes such players are called *dummies*).

There are other apparent anomalies in weighted voting games that are worth mentioning. For example, suppose we take a weighted voting game, and *add* a new voter (keeping the quota unchanged). Now, you might expect that this would *reduce* the power of a voter from the original game, but this is not always the case. Recall the following weighted voting game, in which the final player has no power:

$$(10; 6, 4, 2).$$

Now, suppose we add an agent, as follows:

$$(10; 6, 4, 2, 8).$$

The coalition of the final two players is clearly winning, and the third player no longer has a Shapley value of 0! (This does not prevent the first two from also being winning: in the terminology we introduced above, in the general context of yes/no voting games, this game is not zero sum.)

Turning to the core, we get some positive results: checking whether the core of a weighted voting game is non-empty can be done in polynomial time. In fact, the core of a weighted voting game is non-empty iff *there is an agent present in every winning coalition*. Checking for such an agent is easy: suppose we want to check whether i is present in every winning coalition: draw up a list C of all the agents, except i, that have positive weights. Now, if i is supposed to be present in all winning coalitions, then C must be losing, and adding i to C must make them winning. So, all we have to do is check that $\sum_{j \in C} w_j < q$ and $\sum_{j \in C \cup \{i\}} w_j \geq q$, which are computationally easy checks.

VETO PLAYER

Now, as we noted above, weighted voting games are not a *complete* representation for simple games: that is, some simple games cannot be represented using weighted voting games. However, there is a natural extension to weighted voting games, called *k-weighted voting games*, that *is* complete. The idea of k-weighted voting games is that we define a number of different weighted voting games (with the same set of players), and we then say that a coalition is winning overall if it wins in each of the component games. The 'k' in the name here refers to the number of individual weighted voting games: we can think of a k-weighted voting game as a *conjunction* of weighted voting games.

It is important to note that k-weighted voting games are not theoretical constructs: they feature quite prominently in our lives. For example, the following political systems can be understood as k-weighted voting games [Taylor, 1995; Taylor and Zwicker, 1993]:

- The United States Federal system is a weighted voting system, in which the components correspond to the two chambers (the House of Representatives and the Senate). The players are the president, vice-president, senators, and representatives. In the game corresponding to the House of Representatives, senators have 0 weight, while in the game corresponding to the Senate, representatives have 0 weight. The president is the only player to have non-zero weight in *both* games.
- The voting system of the enlarged European Union is a three-weighted voting game [Bilbao et al., 2002]. Roughly, the idea is that to get a law passed in the enlarged European Union requires the support of a majority of the countries, a majority of the population of the European Union, and a majority of the 'commissioners' of the EU. Each member state is a player, so (as at 2008) the players are:

Germany, the UK, France, Italy, Spain, Poland, Romania, the Netherlands, Greece, Czech Republic, Belgium, Hungary, Portugal, Sweden, Bulgaria, Austria, Slovak Republic, Denmark, Finland, Ireland, Lithuania, Latvia, Slovenia, Estonia, Cyprus, Luxembourg, Malta.

The three component games in the enlarged EU voting system are shown in Figure 13.4. Weights in the first game are assigned according to the number of commissioners that the respective member state has. The second game is a simple majority game: every member state gets one vote, and a law must have the support of at least 14 member states. In the third game, weights are assigned proportionally to the population of the respective member state.

Figure 13.4: Voting in the enlarged European Union is a three-weighted voting game.

$$g_1 = \langle 255; 29, 29, 29, 29, 27, 27, 14, 13, 12, 12, 12, 12, 12, 10, 10, 10, 7, 7, 7, 7, 7, 4, 4, 4, 4, 4, 3\rangle$$
$$g_2 = \langle 14; 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1\rangle$$
$$g_3 = \langle 620; 170, 123, 122, 120, 82, 80, 47, 33, 22, 21, 21, 21, 21, 18, 17, 17, 11, 11, 11, 8, 8, 5, 4, 3, 2, 1, 1\rangle$$

Now, k-weighted voting games are a *complete* representation for simple games: we can represent every simple game as a k-weighted voting game. An interesting question, however, is *how large does k have to be to represent a given simple game?* The smallest number of components required to represent a simple game is referred to as the *dimension* of the game. The dimension of a k-weighted voting game can be understood as one measure of the inherent complexity of the game. Now, it is possible to prove that there are simple coalitional games of dimension exponential in the number of players (i.e. which would require at least 2^n distinct weighted voting systems to represent them). In fact, there are simple examples of coalitional games that are of dimension 2^n. For example, consider a game in which a coalition is winning iff it contains an odd number of players [Taylor and Zwicker, 1993]. On the other hand, this is also a *worst* case: every simple game can be represented by a k-weighted voting game in which k is at most exponential in the number of players. The complexity of decision problems in k-weighted voting games was considered in [Elkind et al., 2008]; for example, checking whether a given k-weighted voting game is 'minimal' (whether it could be represented by a smaller number of components) is NP-complete. It is important to note that questions such as minimality are not an artificial concern: for example, [Bilbao et al., 2002] studied the voting system of the enlarged EU, and showed that, in fact, it is *not* minimal: roughly, one can eliminate either the first or third component. They also showed that there are anomalies with

respect to the voting power of member states. Germany, for example, has much lower power than its size would suggest, while smaller member states such as Luxembourg have greater power than is suggested by their size. (Among other things, this shows that simply allocating weights according to population size does not give power directly corresponding to population size.)

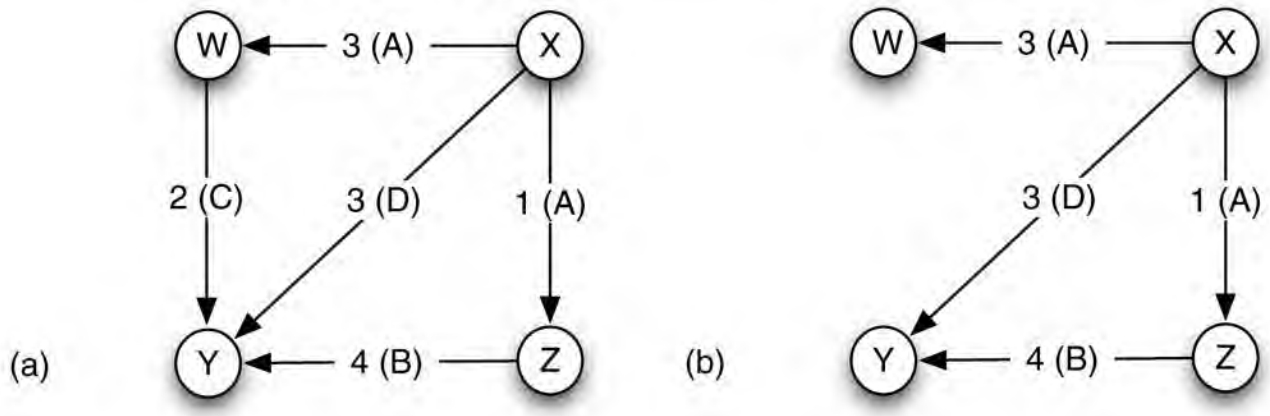
GAME DIMENSION

13.4.2 Network flow games

One reason why weighted voting games are interesting is that they make it clear where the characteristic function of the game is derived from, and have an obvious and meaningful real-world interpretation. *Network flow games* are another interesting class of simple games, which have a similarly compelling real-world interpretation [Bachrach and Rosenschein, 2007]. The idea is as follows. Suppose we have a directed graph, in which edges in the graph are associated with *capacities*, corresponding to the 'bandwidth' of the edge. Such a graph is called a *network flow graph*, and is a very simple and natural model of many real-world networks. For example, we might interpret the model as being one of computer networks, in which case nodes in the graph correspond to computers or network routers, edges correspond to network connections, and capacities to the number of bits per second that can be transmitted along the connection. Alternatively, the network flow might correspond to a road network, in which case nodes correspond to intersections, and capacities might be the number of vehicles per hour. We assume that every edge is controlled by an agent. For example, this might be the department in the organization that is responsible for maintaining the corresponding network link. Figure 13.5a illustrates such a network.

NETWORK FLOW GAME

Figure 13.5: Interpreting simple coalitional games on network flow graphs: (a) nodes in the graph ($\{W, X, Y, Z\}$) correspond to locations, while edges have a bandwidth and an owner – for example, the notation 3(D) on the edge (X, Y) indicates that the edge is owned by D and has a bandwidth of 3; (b) a coalition, in this case $\{A, B, D\}$, induces a subgraph.



If N is such a network, and $C \subseteq Ag$ is a set of agents, then we denote by $N \downarrow C$ the network flow in N that is completely owned by the agents in C – for example, Figure 13.5b shows the network flow owned by the coalition $\{A, B, D\}$.

Now, suppose that we have two locations in the network (say l_1 and l_2), and we want to ensure a bandwidth of b between these locations. Given this setting, we can define a simple game as follows: coalition C gets a value of 1 if it can ensure a flow of at least b from location l_1 to location l_2, otherwise it gets a value of 0. More formally, we have the following characteristic function:

$$v(C) = \begin{cases} 1 & \text{if } N \downarrow C \text{ allows a flow of } b \text{ from } l_1 \text{ to } l_2 \\ 0 & \text{otherwise.} \end{cases}$$

Now, how do we interpret solution concepts in this setting? Measures like the Shapley value can be used here, for example, as a rational basis on which to allocate the maintenance and support budget. If a player corresponding to a particular edge has a very high power, then this indicates that they are extremely important with respect to the goal of ensuring the network flow, and should be allocated a correspondingly high maintenance or support budget.

With respect to the issue of computing Shapley-like values, the general case is computationally very hard; however, [Bachrach and Rosenschein, 2007] identify cases where the network structure is such that these values can be easily computed.

13.5 Coalitional Games with Goals

A basic assumption in the coalitional games considered above is that utility is represented as some numeric value, and that this value can be divided arbitrarily among agents. However, as we saw in Chapter 2, utilities are often not an appropriate way of communicating our desires to software agents. Instead, we frequently have some *goal* to be accomplished, and either the goal is satisfied or it is not. *Qualitative coalitional games* (QCGs) are a type of coalitional game in which each agent has a set of goals, and wants one of them to be achieved (but it doesn't care which) [Wooldridge and Dunne, 2004]. Agents cooperate in QCGs to achieve mutually satisfying sets of goals. To model such scenarios, we assume there is some overall set of possible goals G, and every agent $i \in Ag$ has some set of goals $G_i \subseteq G$ associated with it. The idea is that the agent wants one of these goals to be achieved but does not care which one. To model cooperative action, we assume that every coalition C has a *set of choices,* $V(C)$, associated with it, representing the different ways that the coalition C could choose to cooperate. Formally, the characteristic function for a QCG has the signature:

QUALITATIVE COALITIONAL GAME

$$V: 2^{Ag} \rightarrow 2^{2^G}.$$

Suppose that a set of goals $G' \subseteq G$ is achieved. Then G' will be said to *satisfy* an agent i if $G_i \cap G' \neq \emptyset$, that is, if agent i gets at least one of its goals achieved. (Notice that an agent has no preferences over goals – it simply wants at least one to be achieved.) A goal set G' will be said to be *feasible* for coalition C if G' is one of the choices available to C, that is, if $G' \in V(C)$. Finally, a coalition is *successful* if C can cooperate in such a way that all of its members are satisfied; more formally, C is successful if there is some feasible goal set G' for C such that G'

satisfied. More formally, C is successful if there is some feasible goal set G' for C such that G' satisfies every member of C.

QCGs seem a natural framework within which to study cooperation in goal-oriented systems, but when considering their computational aspects, a by-now familiar problem arises: how do we represent the function V? [Wooldridge and Dunne, 2004] propose a representation based on propositional logic; this representation is complete, but not always guaranteed to be succinct, although it 'often' permits characteristic functions to be represented succinctly. The idea is perhaps best illustrated by example. Suppose we have a system in which goal g_1 can be achieved by a coalition C iff C contains agent 1 and either 2 or 3 or both. Then we can represent this characteristic function via the following propositional logic formula:

$$g_1 \leftrightarrow (1 \wedge (2 \vee 3)).$$

A bit more formally, to represent characteristic functions V, we define a propositional formula, in which Boolean variables in the formula correspond to agents and goals. The idea is that $G' \in V(C)$ iff the formula φ, supposedly representing V, evaluates to true under the valuation that makes the Boolean variables corresponding to G' and C true, and all other variables false. [Wooldridge and Dunne, 2004] investigates the complexity of a range of decision problems for QCGs, using this representation. For example, an interesting possibility in the setting of QCGs is the notion of a *veto player*: we say i is a veto player for j if i is a member of every coalition that can achieve one of i's goals. If i is a veto player for j, then we can say that j is *dependent* on i. We can then generalize this concept to *mutual dependence*, where every player in a coalition is a veto player for every other player, and so on.

Of course, QCGs have nothing to say about where the characteristic function *comes from*, or how it is *derived* for a given scenario. The framework of *coalitional resource games* (CRGs) gives one answer to this question, based on the simple idea that to achieve a goal requires the consumption, or expenditure, of some resources, and that each agent is *endowed* with a profile of resources [Wooldridge and Dunne, 2006]. Then, coalitions will form in order to pool resources, to achieve a mutually satisfactory set of goals. Some interesting questions that then arise relate to resource consumption. For example, suppose we are given a particular budget of resources and we are asked whether a pair of coalitions can achieve their goals (i.e. whether both can be simultaneously successful), staying within the stated resource bounds. This kind of question arises, for example, when setting targets for pollution control: can some countries achieve their economic objectives without consuming too many pollution-producing resources?

COALITIONAL RESOURCE GAME

13.6 Coalition Structure Formation

So far in this chapter, we have implicitly assumed that an agent will act strategically, as in non-cooperative games, attempting to maximize its own utility. However, as we have discussed previously, if the entire system is 'owned' by a single designer, then the performance of individual agents is perhaps not paramount. Instead, we might typically be concerned with *maximizing the social welfare* of the system; that is, maximizing the sum of the values of the individual coalitions. We consider this problem to be one of *coalition structure generation*. A coalition structure is a partition of the overall set of agents Ag into mutually disjoint coalitions. Consider the following example. We have three agents, $Ag = \{1, 2, 3\}$; then there are seven possible coalitions:

$$\{1\}, \{2\}, \{3\}, \{1,2\}, \{2,3\}, \{3,1\}, \{1,2,3\}$$

and five possible coalition structures:

$$\{\{1\}, \{2\}, \{3\}\}, \{\{1\}, \{2,3\}\}, \{\{2\}, \{1,3\}\}, \{\{3\}, \{1,2\}\}, \{\{1, 2,3\}\}.$$

Given a coalitional game $G = (Ag, v)$, we say that the *socially optimal coalition structure* CS^* with respect to G is given as follows:

$$CS^* = \arg \max_{CS \in \text{partitions of } Ag} V(CS)$$

where

$$V(CS) = \sum_{C \in CS} v(C).$$

Unfortunately, there will be exponentially many coalition structures in the number of agents: in fact, a little combinatorial mathematics shows that there will be exponentially *more* coalition structures over the set of agents Ag than there will be coalitions over Ag. So, searching through the space of *all possible* coalition structures to find the optimal coalition structure is extremely undesirable, and indeed infeasible in the worst case. So, can we do better? [Sandholm et al., 1999] studied this problem, and developed a technique that would be guaranteed to find a coalition structure that was within some *provable bound* from the optimal one. The idea is best explained with reference to an example. Suppose we have a system with four agents, $Ag = \{1, 2, 3, 4\}$. There are 15 possible coalition structures for this set of agents; we can usefully visualize these in a *coalition structure graph* – see Figure 13.6. Each node in the graph represents a different possible coalition structure. At level 1 in this graph, we have all the possible coalition structures that contain exactly one coalition: of course, there is just one possible coalition structure containing a single coalition:

COALITION STRUCTURE GRAPH

$$\{\{1,2,3,4\}\}.$$

At level 2 in the graph, we have all the possible coalition structures that contain exactly two coalitions – that is, in level 2, we have all possible ways of partitioning the set of agents $\{1, 2, 3, 4\}$ into two disjoint sets. Then, at level 3, we have the possible coalition structures containing three coalitions, and so on. (Before proceeding, convince yourself that the graph in Figure 13.6 does indeed capture this information faithfully.) An upward edge in the graph represents the division of a coalition in the lower node into two separate coalitions in the upper node. For example, consider the node with the coalition structure

$$\{\{1\}, \{2, 3, 4\}\}.$$

There is an edge from this node to the one with coalition structure

$$\{\{1\}, \{2\}, \{3,4\}\},$$

the point being that this latter coalition structure is obtained from the former by dividing the coalition $\{2, 3, 4\}$ into the coalitions $\{2\}$ and $\{3, 4\}$.

The optimal coalition structure CS^* lies somewhere within the coalition structure graph, and so, to find this, it seems that we would have to evaluate every node in the graph. But consider the bottom two rows of the graph – levels 1 and 2. Observe that every possible coalition (excluding the empty coalition) appears in these two levels. (Of course, not every possible

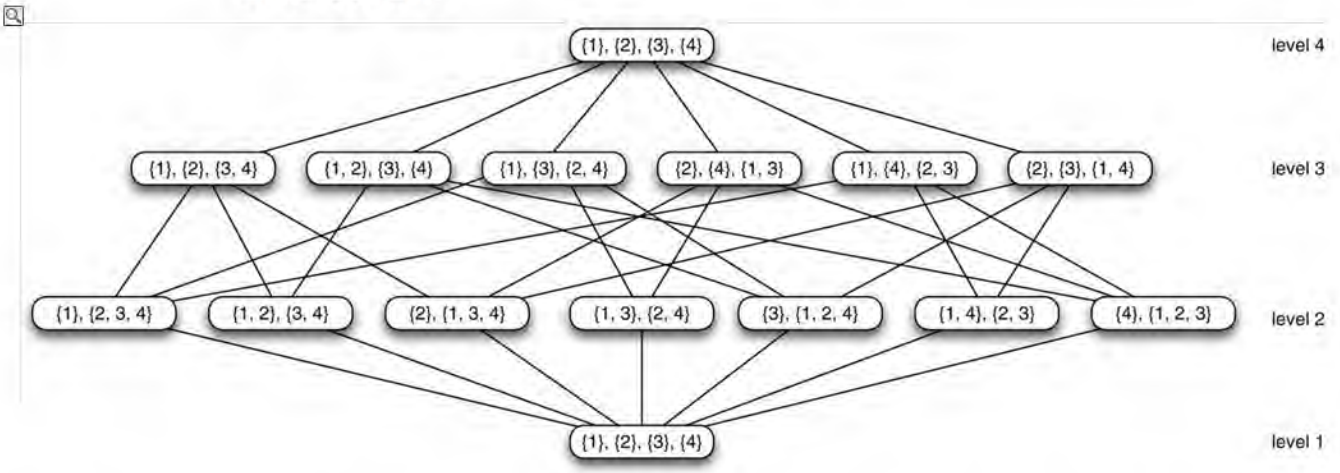
coalition structure appears in these two levels.) Now, suppose we restrict our search for a possible coalition structure to *just* these two levels – we go no higher in the graph. Let CS' be the best coalition structure that we find in these two levels, and let CS^* be the best coalition structure overall. Let C^* be the coalition with the highest value of all possible coalitions:

$$C^* = \arg\max_{C \subseteq Ag} v(C).$$

The value of the best coalition structure we find in the first two levels of the coalition structure graph must be at least as much as the value of the best possible coalition: $V(CS') \geq v(C^*)$. This is because every possible coalition appears in at least one coalition structure in the first two levels of the graph. So assume the worst case, i.e. $V(CS') = v(C^*)$.

Compare the value of $V(CS')$ to $V(CS^*)$. Since $V(CS')$ is the highest possible value of any coalition structure, and there are only n agents ($n = 4$ in the case of Figure 13.6), then the highest possible value of $V(CS^*)$ would be $nv(C^*) = n \cdot V(CS')$. In other words, in the worst possible case, the value of the best coalition structure we find in the first two levels of the graph would be at worst $\frac{1}{n}$ the value of the best, where n is the number of agents; it cannot be any worse than this. Thus, although searching the first two levels of the graph does not guarantee to give us the *optimal* coalition structure, it *does* guarantee to give us one that is no worse that $\frac{1}{n}$ of the optimal.

Figure 13.6: The coalition structure graph for $Ag = \{1, 2, 3, 4\}$. Level 1 has coalition structures containing a single coalition; level 2 has coalition structures containing two coalitions, and so on.



This idea immediately gives us a simple algorithm for finding coalition structures: search the bottom two levels of the coalition structure graph, keeping track of the best coalition structure found so far.

Suppose we have more time available, to continue the search for a good structure beyond the bottom two levels of the graph. How should we structure this search in the best way possible? [Sandholm et al., 1999, p. 218] propose the following algorithm:

1. Search the bottom two levels of the coalition structure graph, keeping track of the best coalition structure seen so far.
1. Now continue with a breadth-first search starting at the *top* of the graph, again keeping

track of the best coalition structure seen so far, and continue until either we have no remaining time, or else we have considered all of the graph not studied in stage (1).

3. Return the coalition structure with the highest value seen.

These techniques were refined and extended in [Rahwan, 2007].

Notes and Further Reading

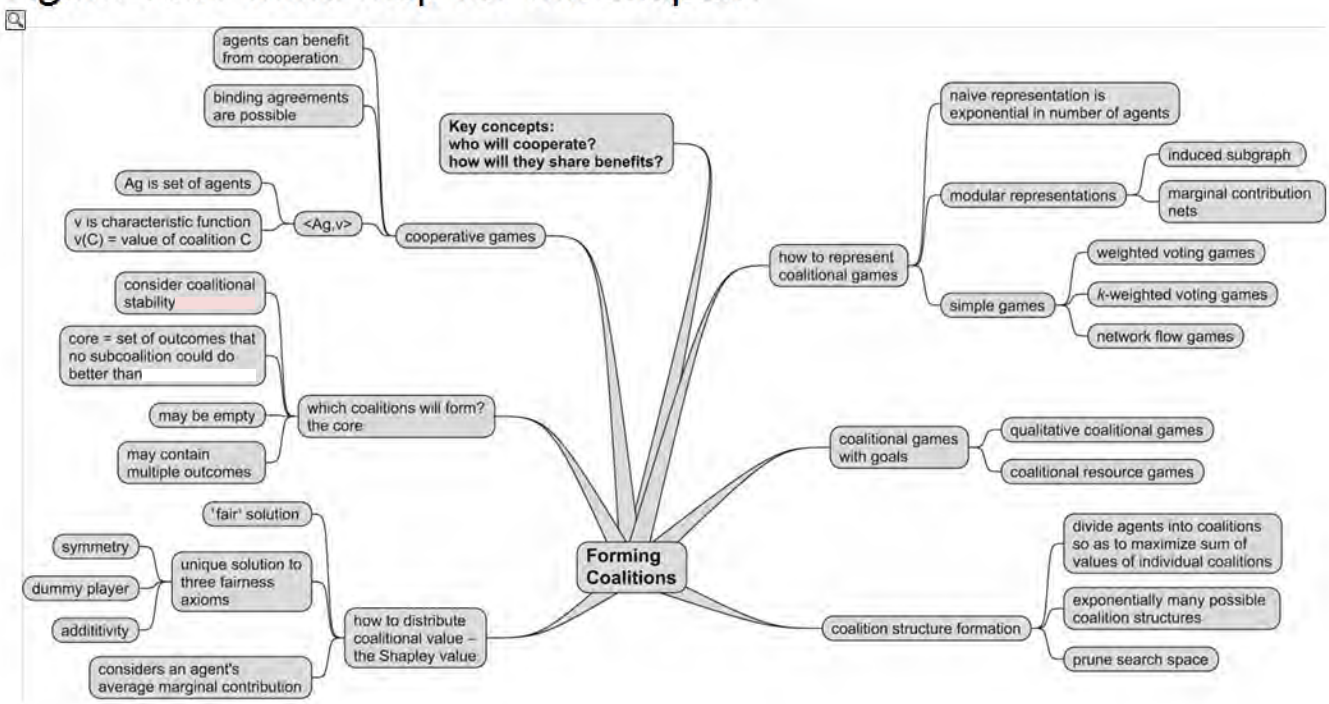
A comprehensive introduction to the theory of cooperative games is presented in [Peleg and Sudholter, 2002]. My treatment is derived from that of [Osborne and Rubinstein, 1994].

Cooperative games were introduced to the multiagent systems community largely through the work of Onn Shehory and Sarit Kraus, who developed algorithms for coalition structure formation in which agents were modelled as having different capabilities, and were assumed to benevolently desire some overall task to be accomplished, where this task had some complex (plan-like) structure [Shehory and Kraus, 1995a, b, 1998]. Samuel Ieong's PhD thesis gives detailed discussions on the subject of representation in coalitional games [Ieong, 2008].

[Taylor and Zwicker, 1999] is a thorough guide to simple games in general, and weighted voting games in particular; k-weighted voting games, and the dimensionality of such games, are studied in [Taylor and Zwicker, 1993]. The complexity of weighted voting games was studied in [Deng and Papadimitriou, 1994], and revisited in [Elkind et al., 2007, 2008].

Class reading: [Ieong and Shoham, 2005]. This is a technical article (but a very nice one!), introducing the marginal contribution nets scheme.

Figure 13.7: Mind map for this chapter.



Chapter 14 Allocating Scarce Resources

Auctions used to be comparatively rare in everyday life; every now and then, one would hear of astronomical sums paid at auction for a painting by Monet or Van Gogh, but otherwise, they did not enter the lives of the majority. The Internet and the Web fundamentally changed this. The Web made it possible for auctions with a large, international audience to be carried out at very low cost. This in turn made it possible for goods to be put up for auction which hitherto would have been too uneconomical. Large businesses have sprung up around the idea of online auctions, with eBay being perhaps the best-known example [eBay, 2001].

Fundamentally, auctions are mechanisms used to reach agreements on one very simple issue: that of *how to allocate scarce resources to agents*. The notion of a 'resource' here is very general. The resource could be a painting, as in classic Sotheby's-style auctions; it could be the right to exploit an area of land for its mineral resources; it could be the right to sell services that use part of the electromagnetic spectrum; or it could be some processor cycles on your PC. The point is that the resource in question is scarce, and is typically desired by more than one agent. If the resource isn't scarce, then there is no problem in allocating it; if it isn't desired by more than one agent, then again there is no problem in allocating it. But the truth is that, in general, resources *are* scarce, and they are desired by more than one agent. The question of what is a reasonable way to allocate resources then arises. Auctions provide principled ways to allocate them to agents. In particular, auctions are effective at allocating resources *efficiently*, in the sense of allocating resources to those that value them the most. As we will see, certain types of auctions also have the property that they can, almost magically, reveal hidden truths about bidders. We will start by introducing the basic terminology and concepts of auctions, and then consider the simplest kinds of auctions, some of which you will almost certainly be familiar with. We will then consider the technically more complex combinatorial auctions, and conclude by looking at one of the most interesting contributions of auction theory: the area of *mechanism design*.

RESOURCE ALLOCATION

14.1 Classifying Auctions

Abstractly, an auction takes place between an agent known as the *auctioneer* and a collection of agents known as the *bidders*. The goal of the auction is for the auctioneer to allocate a '*good*' (i.e. a scarce resource) to one of the bidders. Certain types of auctions are concerned with situations where there is more than one good, but for the moment we will focus on the simple case. In most settings – and certainly most traditional auction settings – the auctioneer desires to maximize the price at which the good is allocated, while bidders desire to minimize the selling price. The auctioneer will attempt to achieve their desire through the design of an appropriate auction, while bidders attempt to achieve their desires by using an effective strategy.

There are several factors that can affect both the auction protocol and the strategy that agents use. The most important of these is whether the good for auction has a *private* or a *public/common* value. Consider an auction for a one-dollar bill. How much is this dollar bill worth to you? Assuming it is a 'typical' dollar bill, then it should be worth exactly $1; if you paid $2 for it, you would be $1 worse off than you were. The same goes for anyone else involved in this auction. A typical dollar bill thus has a *common value:* it is worth exactly the

same to all bidders in the auction. However, suppose you were a big fan of the Beatles, and the dollar bill happened to be the last dollar bill that John Lennon spent. Then it may well be that, for sentimental reasons, this dollar bill was worth considerably more to you – you might be willing to pay $100 for it. To a fan of the Rolling Stones, with no interest in or liking for the Beatles, however, the bill might not have the same value. Someone with no interest in the Beatles whatsoever might value the one-dollar bill at exactly $1. In this case, the good for auction – the dollar bill – is said to have a *private value*: each agent values it differently.

COMMON VALUE

PRIVATE VALUE

A third type of valuation is *correlated value*: in such a setting, an agent's valuation of the good depends partly on private factors, and partly on other agents' valuations of it. An example might be where an agent was bidding for a painting that it liked, but wanted to keep open the option of later selling the painting. In this case, the amount you would be willing to pay would depend partly on how much you liked it, but also partly on how much you believed other agents might be willing to pay for it if you put it up for auction later.

CORRELATED VALUE

Let us turn now to consider some of the dimensions along which auction protocols may vary. The first is that of *winner determination*: who gets the good that the bidders are bidding for, and what do they pay? In the auctions with which we are most familiar, the answer to this question is probably self-evident: the agent that bids the most is allocated the good, and pays the amount of the bid. Such protocols are known as *first-price* auctions. This is not the only possibility, however. A second possibility is to allocate the good to the agent that bid the highest, but this agent pays only the amount of the *second* highest bid. Such auctions are known as *second-price* auctions. At first sight, it may seem bizarre that there are any settings in which a second-price auction is desirable, as this implies that the auctioneer does not get as much for the good as it could do. However, we shall see below that there are indeed some settings in which a second-price auction is desirable.

FIRST-PRICE AUCTION

SECOND-PRICE AUCTION

The second dimension along which auction protocols can vary is whether or not the bids made by the agents are known to each other. If every agent can see what every other agent is bidding (the terminology is that the bids are *common knowledge*), then the auction is said to be *open cry*. If the agents are not able to determine the bids made by other agents, then the auction is said to be a *sealed-bid* auction.

OPEN CRY

SEALED BID

A third dimension is the mechanism by which bidding proceeds. The simplest possibility is to have a single round of bidding, after which the auctioneer allocates the good to the winner

have a single round of bidding, after which the auctioneer allocates the good to the winner. Such auctions are known as *one shot*. The second possibility is that the price starts low (often at a *reservation price*) and successive bids are for increasingly large amounts. Such auctions are known as *ascending*. The alternative – *descending* – is for the auctioneer to start off with a high value, and to decrease the price in successive rounds.

ONE-SHOT AUCTION

ASCENDING AUCTION

14.2 Auctions for Single Items

Given the classification scheme above, we can identify a wide range of different possible auction types. In this section, we will look at the simplest of these, which concern the allocation of just a single item.

14.2.1 English auctions

English auctions are the most commonly known type of auction, made famous by such auction houses as Sotheby's. English auctions are *first-price, open cry, ascending* auctions:

ENGLISH AUCTIONS

- The auctioneer starts off by suggesting a *reservation price* for the good (which may be 0) – if no agent is willing to bid more than the reservation price, then the good is allocated to the auctioneer for this amount.
- Bids are then invited from agents, who must bid more than the current highest bid – all agents can see the bids being made, and are able to participate in the bidding process if they so desire.
- When no agent is willing to raise the bid, then the good is allocated to the agent that has made the current highest bid, and the price they pay for the good is the amount of this bid.

What strategy should an agent use to bid in English auctions? It turns out that the dominant strategy is for an agent to successively bid a small amount more than the current highest bid until the bid price reaches their private valuation, and then to withdraw.

Simple though English auctions are, it turns out that they have some interesting properties. One interesting feature of English auctions arises when there is uncertainty about the true value of the good being auctioned. For example, suppose an auctioneer is selling some land to agents that want to exploit it for its mineral resources, and that there is limited geological information available about this land. None of the agents thus knows exactly what the land is worth. Suppose now that the agents engage in an English auction to obtain the land, each using the dominant strategy described above. When the auction is over, should the winner feel happy that they have obtained the land for less than or equal to their private valuation? Or should they feel worried *because no other agent valued the land so highly?* This situation, where the winner is the one who overvalues the good on offer, is known as the *winner's curse*. Its occurrence is not limited to English auctions, but it occurs most frequently in these.

WINNER'S CURSE

14.2.2 Dutch auctions

Dutch auctions are examples of *open-cry descending* auctions:

DUTCH AUCTION

- The auctioneer starts out offering the good at some artificially high value (above the expected value of any bidder's valuation of it).
- The auctioneer then continually lowers the offer price of the good by some small value, until some agent makes a bid for the good which is equal to the current offer price.
- The good is then allocated to the agent that made the offer.

Notice that Dutch auctions are also susceptible to the winner's curse.

14.2.3 First-price sealed-bid auctions

First-price sealed-bid auctions are examples of one-shot auctions, and are perhaps the simplest of all the auction types we will consider. In such an auction, there is a single round, in which bidders submit to the auctioneer a bid for the good; there are no subsequent rounds, and the good is awarded to the agent that made the highest bid. The winner pays the price of the highest bid. There are hence no opportunities for agents to offer larger amounts for the good.

How should an agent act in first-price sealed-bid auctions? Suppose every agent bids their true valuation; the good is then awarded to the agent that bid the highest amount. But consider the amount bid by the second highest bidder. The winner could have offered just a tiny fraction more than the second highest price, and still been awarded the good. Hence most of the difference between the highest and second highest price is, in effect, money wasted as far as the winner is concerned. The best strategy for an agent is therefore to bid less than its true valuation. How *much* less will of course depend on what the other agents bid – there is no general solution.

14.2.4 Vickrey auctions

The next type of auction is the most unusual and perhaps most counterintuitive of all the auction types we shall consider. Vickrey auctions are *second-price sealed-bid* auctions. This means that there is a single bidding round, during which each bidder submits a single bid; bidders do not get to see the bids made by other agents. The good is awarded to the agent that made the highest bid; however, the price this agent pays is not the price of the highest bid, but the price of the *second-highest* bid. Thus if the highest bid was made by agent i, who bid \$9, and the second highest bid was by agent j, who bid \$8, then agent i would win the auction and be allocated the good, *but agent i would only pay* \$8.

VICKREY AUCTION

Why would one even consider using Vickrey auctions? The answer is that Vickrey auctions make truth telling the dominant strategy: *a bidder's dominant strategy in a private value Vickrey auction is to bid their true valuation*. Using the terminology of Chapter 12, Vickrey auctions are *not prone to strategic manipulation*.

Let us consider why this is.

- Suppose that you bid *more* than your true valuation. In this case, you may be awarded

Suppose that you bid more than your true valuation. In this case, you may be awarded the good, but you run the risk of being awarded the good at more than the amount of your private valuation. If you win in such a circumstance, then you make a loss (since you paid more than you believed the good was worth).

- Suppose you bid *less* than your true valuation. In this case, note that you stand less chance of winning than if you had bid your true valuation. But, even if you do win, the amount you pay will not have been affected by the fact that you bid less than your true valuation, because you will pay the price of the second highest bid.

Thus there is no benefit to doing anything other than bidding to your private valuation in a Vickrey auction – no more and no less.

Because they make truth telling the dominant strategy, Vickrey auctions have received a lot of attention in the multiagent systems literature (see [Sandholm, 1999, p. 213] for references). However, they are not widely used in human auctions. There are several reasons for this, but perhaps the most important is that humans frequently find the Vickrey mechanism hard to understand, because at first sight it seems so counterintuitive.

Note that Vickrey auctions make it possible for *antisocial* behaviour to occur. Suppose you want some good and your private valuation is \$90, but you know that some other agent wants it and values it at \$100. As truth telling is the dominant strategy, you can do no better than bid \$90; your opponent bids \$100, is awarded the good, but pays only \$90. Well, maybe you are not too happy about this: maybe you would like to 'punish' your successful opponent. How can you do this? Suppose you bid \$99 instead of \$90. Then you still lose the good to your opponent – *but they pay \$9 more than they would do if you had bid truthfully*. To make this work, of course, you have to be very confident about what your opponent will bid – you do not want to bid \$99 only to discover that your opponent bid \$95, and you were left with a good that cost \$5 more than your private valuation. This kind of behaviour occurs in commercial situations, where one company may not be able to compete directly with another company, but uses their position to try to force the opposition into bankruptcy.

14.2.5 Expected revenue

There are several issues that should be mentioned relating to the types of auctions discussed above. The first is that of *expected revenue*. If you are an auctioneer, then, as mentioned above, your overriding consideration will in all likelihood be to maximize your revenue: you want an auction protocol that will get you the highest possible price for the good on offer. You may well not be concerned with whether or not agents tell the truth, or whether they are afflicted by the winner's curse. It may seem that some protocols – Vickrey's mechanism in particular – do not encourage this. So, which should the auctioneer choose?

For private value auctions, the answer depends partly on the attitude to risk of both auctioneers and bidders [Sandholm, 1999, p. 214].

- For *risk-neutral bidders*, the expected revenue to the auctioneer is provably identical in all four types of auctions discussed above (under certain simple assumptions). That is, the auctioneer can expect on average to get the same revenue for the good using all of these types of auction.
- For *risk-averse bidders* (i.e. bidders that would prefer to get the good even if they paid slightly more for it than their private valuation), Dutch and first-price sealed-bid protocols lead to higher expected revenue for the auctioneer. This is because, in these

protocols, risk-averse agents can 'insure' themselves by bidding slightly more for the good than would be offered by a risk-neutral bidder.

- *Risk-averse auctioneers*, however, do better with Vickrey or English auctions.

Note that these results should be treated very carefully. For example, the first result, relating to the revenue equivalence of auctions given risk-neutral bidders, depends critically on the fact that bidders really do have private valuations. In choosing an appropriate protocol, it is therefore critical to ensure that the properties of the auction scenario – and the bidders – are understood correctly.

14.2.6 Lies and collusion

An interesting question is the extent to which the protocols we have discussed above are susceptible to lying and collusion by both bidders and auctioneer. Ideally, as an auctioneer, we would like a protocol that was immune to collusion by bidders, i.e. that made it against a bidder's best interests to engage in collusion with other bidders. Similarly, as a potential bidder in an auction, we would like a protocol that made honesty on the part of the auctioneer the dominant strategy.

None of the four auction types discussed above is immune to collusion. For any of them, the 'grand coalition' of all agents involved in bidding for the good can agree beforehand to collude to put forward artificially low bids for the good on offer. When the good is obtained, the bidders can then obtain its true value (higher than the artificially low price paid for it), and split the profits among themselves. The most obvious way of preventing collusion is to modify the protocol so that bidders cannot identify each other. Of course, this is not popular with bidders in open-cry auctions, because bidders will want to be sure that the information they receive about the bids placed by other agents is accurate.

With respect to the honesty or otherwise of the auctioneer, the main opportunity for lying occurs in Vickrey auctions. The auctioneer can lie to the winner about the price of the second highest bid, by overstating it and thus forcing the winner to pay more than they should. One way around this is to 'sign' bids in some way (e.g. through the use of a digital signature), so that the winner can independently verify the value of the second highest bid. Another alternative is to use a trusted third party to handle bids. In open-cry auction settings, there is no possibility for lying by the auctioneer, because all agents can see all other bids; first-price sealed-bid auctions are not susceptible because the winner will know how much they offered.

Another possible opportunity for lying by the auctioneer is to place bogus bidders, known as *shills*, in an attempt to artificially inflate the current bidding price. Shill bidding is a common complaint in online auctions such as eBay [Gregg and Scott, 2008, p. 70].

14.2.7 Counterspeculation

Before we leave auctions, there is at least one other issue worth mentioning: that of *counterspeculation*. This is the process of a bidder engaging in an activity in order to obtain information either about the true value of the good on offer, or about the valuations of other bidders. Clearly, if counterspeculation were free (i.e. it did not cost anything in terms of time or money) and accurate (i.e. counterspeculation would accurately reduce an agent's uncertainty about either the true value of the good or the value placed on it by other bidders), then every agent would engage in it at every opportunity. However, in most settings, counterspeculation is not free: it may have a time cost and a monetary cost. The time cost will

counterspeculation is not free. It may have a time cost and a monetary cost. The time cost will matter in auction settings (e.g. English or Dutch) that depend heavily on the time at which a bid is made. Similarly, investing money in counterspeculation will only be worth it if, as a result, the bidder can expect to be no worse off than if it did not counterspeculate. In deciding whether to speculate, there is clearly a trade-off to be made, balancing the potential gains of counterspeculation against the costs (money and time) that it will entail. (It is worth mentioning that counterspeculation can be thought of as a kind of meta-level reasoning, and the nature of these trade-offs is thus very similar to that of the trade-offs in practical reasoning agents as discussed in earlier chapters.)

14.3 Combinatorial Auctions

The auction types we presented above all assume that the resources in question are basically simple things, with no *structure*. That is, it is assumed that we are auctioning a *single, indivisible* good. In many situations, this is not the case, however. There may be many goods; moreover, some of these goods may be identical, while others are different. The bidders will typically have preferences over the possible *bundles* of goods. For example, consider auctioning mobile phone bandwidth, in the sense of the 3G spectrum auctions that took place across the developed world in the early part of the 21st century. Here, a government was auctioning off licences for a range of frequency bands, in a range of geographic locations. For example, one single licence (i.e. one of the goods being auctioned) might cover the 1.7 GHz to 1.72 GHz bandwidth for a geographical area corresponding to the city of Liverpool. A different licence might cover the 1.72 GHz to 1.74 GHz bandwidth for Liverpool. Now, the preferences of a company buying such licences might derive from considerations such as the fact that all the services to be provided would use a single bandwidth, or from the fact that geographically linked licences make sense for service providers, and so on (see sidebar *The Biggest Auction Ever*).

To make these ideas a bit more precise, let $Z = \{z_1, \ldots, z_m\}$ be a set of items to be auctioned. We capture the preferences of each agent $i \in Ag$ via a *valuation function*:

VALUATION FUNCTION

$$v_i : 2^Z \rightarrow \mathbb{R}$$

The Biggest Auction Ever

When the use of mobile phones exploded in the 1990s, governments across the world were delighted to find that they had a new resource that they could make money from: *bandwidth* – the electromagnetic frequencies used by mobile phones to send their signals. Governments decided to sell licences to exploit this bandwidth to the providers of mobile phone services. The way that many governments chose to do this, however, was quite novel at the time. They decided to sell the licences by auction. These 'spectrum auctions' were typically complex combinatorial auctions, with multiple frequencies and geographical locations. In the UK, the government solicited the assistance of game theorists Ken Binmore and Paul Klemperer to design the auctions [Binmore and Klemperer, 2002]. The government stated its aims as being to assign the spectrum efficiently, to promote competition, and to 'realize the full economic value' of the spectrum (i.e. to maximize revenue) [Binmore and Klemperer, 2002]. Based on advice from Binmore and Klemperer, the UK chose to adopt a *simultaneous*

ascending auction model. In this model, bidding takes place in a series of rounds, with bidders able to bid for one licence in a round; at the end of the round, the highest bid for a licence was incremented by an amount chosen by the auctioneer, and this value was then set to be the minimum bid for that licence in the next round. Bidders were required to stay 'active' in bidding, or else drop out. The process continued until no bidders were willing to increase their bid. The auctions took place between 6 March and 27 April 2000; there were 150 rounds in total, with 13 companies participating. In the end, the five licences were sold for between £4 billion and £6 billion, netting the UK government a cool £22.5 billion (US$ 34 billion) revenue. To put this figure into context, it represents about £570 per person in the UK, or about 2.5% of the gross national product of the UK (how much the UK earns in a year). However, the auctions took place at more-or-less exactly the peak of the 'dot com' boom, when there was massive (and frequently absurd) speculative investment in computer and communications technologies. This market sector shrank rapidly from March 2000 onwards, and it seems many of the winners in the auctions regretted their participation. There were claims that so much had been paid for the licences that the financial viability of the winners had been compromised: 'there was a great deal of caterwauling as telecom executives sought to blame … game theorists who supposedly made them pay more for their licences than they were worth. But who but an idiot would bid more for something than they thought it was worth?' [Binmore, 2007, p. 107].

so that, for every bundle of goods $Z \subseteq Z$, the value $v_i(Z)$ indicates how much Z would be worth to agent i. It is worth identifying some sensible properties of valuation functions.

Normalization A valuation function $v_i(\ldots)$ is said to be normalized if $v_i(\varnothing) = 0$, i.e. if agent i does not get any value from an empty allocation.

Free disposal The idea of free disposal is that an agent is *never worse off for having more goods*. Formally, this constraint is expressed as follows:

FREE DISPOSAL

$$Z_1 \subseteq Z_2 \text{ implies } v_i(Z_1) \leq v_i(Z_2).$$

An *outcome* for a combinatorial auction is an *allocation of goods being auctioned among the agents*. Notice that we don't require *all* goods to be allocated: the auctioneer could decide not to allocate some (e.g. if no agent is interested in a good, then the auctioneer may decide to reserve it for later auctions). Formally, an allocation is a list of sets $Z_1, \ldots, Z_n$, one for each agent $i \in Ag$, such that $Z_i \subseteq Z$, and for all $i, j \in Ag$ such that $i \neq j$, we have $Z_i \cap Z_j = \varnothing$ (i.e. no good is allocated to more than one agent). Let $alloc(Z, Ag)$ denote the set of all such allocations of goods Z over agents Ag.

In Chapter 12, we saw that different social choice mechanisms (i.e. different voting procedures) satisfied different properties. We can think of a combinatorial auction as a type of social choice mechanism, where the outcomes correspond to the possible allocations of goods to agents, and the preferences that agents have over allocations are given by their valuation functions. In this context, it is interesting to ask what properties (Pareto optimality, …) we would like an auction to satisfy. One very natural idea is that we would like an auction to *maximize social welfare*. Recall from Chapter 11 that the social welfare of an outcome is the sum of utilities obtained in that outcome. Let us define a function $sw(\ldots)$ that gives the social welfare of an allocation in

terms of the values obtained by the agents from that allocation:

$$sw(\underbrace{Z_1, \ldots, Z_n}_{\text{allocation}}, \underbrace{v_1, \ldots, v_n}_{\text{valuations}}) = \sum_{i=1}^{n} v_i(Z_i).$$

Given this setting, we can now describe – at a rather high level – what we mean by a combinatorial auction. We are given a set of goods Z and a collection of valuation functions v_1, …, v_n, one for each agent $i \in Ag$. The goal is to find an allocation Z_1^*, …, Z_n^* that maximizes the function *sw*, i.e.

$$Z_1^*, \ldots, Z_n^* = \arg \max_{(Z_1, \ldots, Z_n) \in alloc(Z, Ag)} sw(Z_1, \ldots, Z_n, v_1, \ldots, v_n).$$

The problem of computing the optimal allocation Z_1^*, …, Z_n^* is called the *winner determination problem*. Now, you might think that this looks like a classic combinatorial optimization problem, and you would be right about that: combinatorial auctions are where auctions meet combinatorial optimization.

WINNER DETERMINATION PROBLEM

So, now we know what we want to do. We want to find an allocation Z_1^*, …, Z_n^* that maximizes social welfare, in the way we just described. But how do we go about this? Remember, we don't have access to agents' actual valuations – this is private information. As a first attempt to come up with a mechanism for obtaining Z_1^*, …, Z_n^*, , you might suggest the following social choice procedure:

1. Every agent $i \in Ag$ simultaneously declares to the mechanism a valuation $\hat{v}_i$ (remember that agent *i*'s *actual* valuation is v_i, but the mechanism doesn't have access to this, and so can only ask an agent to declare a valuation function: it may be that an agent does not declare its true valuation function, and so we write $\hat{v}_i$ to emphasize the fact that it is possible that $\hat{v}_i \neq v_i$).
2. Using valuation functions $\hat{v}_1$, …, $\hat{v}_n$ in its computation of *sw* (…), the mechanism computes

   $$Z_1^*, \ldots, Z_n^* = \arg \max_{(Z_1, \ldots, Z_n) \in alloc(Z, Ag)} sw(Z_1, \ldots, Z_n, \hat{v}_1, \ldots, \hat{v}_n)$$

   and the allocation Z_1^*, …, Z_n^* is then implemented.

The very obvious problem with this mechanism, of course, is that, using the terminology of Chapter 12, it falls prey to strategic manipulation. In fact, it is very easy to manipulate: an agent can simply overstate the value of possible bundles. We will soon see a very elegant mechanism – the VCG mechanism – that deals with this problem. For now, however, we will focus on the computational issues surrounding the *sw* (…) function and the simple mechanism we have just described.

Essentially, there are two basic problems that we need to address in order to be able to

Essentially, there are two basic problems that we need to address in order to be able to implement combinatorial auctions in the manner described [Blumrosen and Nisan, 2007, p. 268]:

> **Representational complexity** Recall that a valuation function v_i has the signature $v_i: 2^Z \rightarrow \mathbb{R}$. As we saw in Chapter 13, a key problem with such functions from a computational point of view is that naive representations for them (a table that lists for every possible set of goods the corresponding value) are exponential in the number of goods. Such representations are completely impractical: for example, a recent bandwidth auction had 1122 licences up for auction, and so a table defining a valuation function for such an auction would require 2^{1122} entries – vastly larger than the number of atomic particles in the universe.
>
> **Computational complexity** As we described above, winner determination is a combinatorial optimization problem, and like most such problems, it is computationally complex. In fact, winner determination is NP-hard even under quite restrictive assumptions [Lehmann et al., 2006, p. 304].

We consider these two issues in more detail in the sections that follow.

14.3.1 Bidding languages

With respect to the first issue, the obvious line of attack is to try to develop succinct representation schemes for valuation functions. We are on familiar territory here (or at least, we are if we read Chapter 13): this is exactly what we did with respect to representing characteristic functions in coalitional games. In combinatorial auctions, these representations are called *bidding languages*.

BIDDING LANGUAGES

Most approaches to developing bidding languages take their inspiration from logic. They start from *atomic bids*. An atomic bid β is a pair (Z, p), where $Z \subseteq Z$ is a set of items, and $p \in \mathbb{R}_+$ is a positive real number which indicates the price the relevant bidder is willing to pay for the goods Z. We say a bundle of goods Z' satisfies an atomic bid (Z, p) if $Z \subseteq Z'$. Thus, for example, if my atomic bid is $(\{a, b\}, 4)$ and you offer a bundle $\{a, b, c\}$, then your offer satisfies my bid, since $\{a, b\} \subseteq \{a, b, c\}$. In contrast, if you proposed a bundle that would give me $\{b, d\}$, then this bundle would not satisfy my bid, since $\{a, b\} \not\subseteq \{b, d\}$; similarly, the bundles $\{a\}$, $\{b\}$, and $\{c\}$ would not satisfy my bid. If I make a bid of (Z, p), and you offer me a bundle that satisfies my bid, then my bid says that the amount I am willing to pay is p.

ATOMIC BIDS

More precisely, an atomic bid $\beta = (Z, p)$ defines a valuation function v_β such that:

$$v_\beta(Z') = \begin{cases} p & \text{if } Z' \text{ satisfies } (Z, p) \\ 0 & \text{otherwise.} \end{cases}$$

Now, if we were just restricted to atomic bids of this type, then we would not be able to express very interesting valuation functions. So, to create more interesting valuation

functions, we *combine them* into *complex* bids using operators derived from logical connectives.

XOR bids

We first consider *exclusive or bids* (*XOR bids*) [Sandholm, 2002]. The idea in an XOR bid is that we specify a number of separate bids, but we will pay for *at most one* of these to be satisfied. If an allocation satisfies more than one component of an XOR bid, then we pay the amount of the largest atomic bid satisfied.

XOR BIDS

For example, consider the following XOR bid [Blumrosen and Nisan, 2007, p. 280]:

$$\beta_1 = (\{a, b\}, 3) \text{ XOR } (\{c, d\}, 5).$$

Informally, bid β_1 expresses the following valuation function:

> I would pay 3 for a bundle that contains *a* and *b* but not *c* and *d;* I'll pay 5 for a bundle that contains *c* and *d* but not *a* and *b*; and I'll pay 5 for a bundle that contains *a, b, c*, and *d*.

Thus we have

$v_{\beta 1}(\{a\})$	=	0
$v_{\beta 1}(\{b\})$	=	0
$v_{\beta 1}(\{a, b\})$	=	3
$v_{\beta 1}(\{c, d\})$	=	5
$v_{\beta 1}(\{a, b, c, d\})$	=	5

Let's have another example:

$$\beta_2 = (\{e, f, g\}, 2) \text{ XOR } (\{e\}, 2) \text{ XOR } (\{c, d, e\}, 4).$$

We have:

$v_{\beta 2}(\{e\})$	=	2
$v_{\beta 2}(\{e, f\})$	=	2
$v_{\beta 2}(\{c, d, f, g\})$	=	0
$v_{\beta 2}(\{a, b, c, d, e, f, g\})$	=	4
$v_{\beta 2}(\{c, d, e\})$	=	4

A bit more formally, suppose β is an XOR bid of the form

$$\beta = (Z_1, p_1) \text{ XOR } \ldots \text{ XOR } (Z_k, p_k).$$

Then such a bid defines a valuation function v_β as follows:

$$v_\beta(Z') = \begin{cases} 0 & \text{if } Z' \text{ does not satisfy any of } (Z_1, p_1), \ldots, (Z_k, p_k) \\ \max\{p_i \mid Z_i \subseteq Z'\} & \text{otherwise.} \end{cases}$$

The first clause says that I'll pay nothing if your allocation Z' does not satisfy any of my atomic bids; the second clause says that otherwise, I'll pay the largest price of any of my satisfied atomic bids.

Now, obviously, XOR bids allow us to express more complex valuation functions than is possible with atomic bids alone. But, in fact, XOR bids are *fully* expressive, in the sense that *any* valuation function over a set of goods Z can be expressed using an XOR bid [Nisan, 2006, p. 220]. However, to represent some valuation functions, we require an XOR bid containing a number of atomic bids that is exponential in the number of goods $|Z|$ [Nisan, 2006, p. 220]. With respect to computational complexity, given an XOR bid β, and a bundle of goods $Z \subseteq Z$, computing the valuation $v_\beta(Z)$ can be done in polynomial time.

OR Bids

In an OR bid, as with XOR bids, we specify a number of separate atomic bids. This time, however, we are willing to pay *for more than one bundle*. The exact meaning of such a bid is a little involved to define, however. Suppose I make an OR bid β as follows:

OR BIDS

$$\beta = (Z_1, p_1) \text{ OR } \ldots \text{ OR } (Z_k, p_k).$$

You then propose an allocation in which I get goods $Z' \subseteq Z$. Then to determine my valuation for Z', we proceed as follows. We find the set of atomic bids W contained in β such that:

- every bid in W is satisfied by Z'
- each pair of bids in W has mutually disjoint sets of goods:

$$Z_i \cap Z_j = \varnothing \quad \text{for all } i, j \text{ such that } i \neq j$$

- there is no other subset of bids W' from W satisfying the first two conditions, such that

$$\sum_{(Z_i, p_i) \in W'} p_i > \sum_{(Z_j, p_j) \in W} p_j .$$

Let's have an example. Suppose β_1 is defined:

$$\beta_1 = (\{a, b\}, 3) \text{ OR } (\{c, d\}, 5).$$

Then we have

$v\beta_1(\{a\})$	=	0
$v\beta_1(\{b\})$	=	0
$v\beta_1(\{a, b\})$	=	3
$v\beta_1(\{c, d\})$	=	5
$v\beta_1(\{a, b, c, d\})$	=	8.

The difference with the XOR example considered above is with the final case, where we indicate that we would be willing to pay $3 + 5 = 8$ for the bundle $\{a, b, c, d\}$. Consider β_3:

$$\beta_3 = (\{e, f, g\}, 4) \text{ OR } (\{f, g\}, 1) \text{ OR } (\{e\}, 3) \text{ OR } (\{c, d\}, 4).$$

We have:

$v\beta_3(\{e\})$	=	3
$v\beta_3(\{e, f\})$	=	3
$v\beta_3(\{c, d, f, g\})$	=	5
$v\beta_3(\{a, b, c, d, e, f, g\})$	=	8
$v\beta_3(\{c, d, e\})$	=	7.

OR bids are strictly less expressive than XOR bids, because they cannot represent all valuation functions. For example, consider a valuation function v where we have:

$$v(\{a\}) = 1 \quad v(\{b\}) = 1 \quad v(\{a, b\}) = 1.$$

It is not hard to see that such a valuation function cannot be represented by an OR bid [Nisan, 2006, p. 220]. However, in some cases, OR bids can be exponentially more succinct than XOR bids [Nisan, 2006, p. 220].

In terms of computational complexity, even quite fundamental problems are NP-hard in the context of OR bids. For example, given an OR bid β and a bundle $Z \subseteq Z$, computing $v_\beta(Z)$ is NP-hard [Nisan, 2006, p. 227].

Other combinations of bids

So far we have looked at just two operators on bids: XOR and OR. Richer bidding languages can of course be constructed using other operators, or using combinations of operators. [Fujishima et al., 1999] introduce a language which neatly extends OR bids with the power of XOR bids. The idea is to introduce *dummy* goods into OR bids. Any bid will satisfy a dummy good, but the semantics of OR bids (that we will only pay for mutually disjoint satisfied bids) enforces exclusion between OR bids. For example, suppose d is a dummy bid: then the XOR bid

DUMMY GOODS

$$(Z_1, p_1) \text{ XOR } (Z_2, p_2)$$

can be represented by the following OR-with-dummy-goods bid:

$$(Z^1 \cup \{d\}, p_1) \text{ OR } (Z_2 \cup \{d\}, p_2).$$

Other variations are discussed in [Nisan, 2006].

14.3.2 Winner determination

We noted above that winner determination is a combinatorial optimization problem: we are trying to find some set of goods that maximizes some valuation function. Like almost all such problems, winner determination is NP-hard. So, how can we overcome this problem? We have two obvious lines of attack.

- First, we can try to develop techniques that are *optimal* (i.e. will always give us the best allocation), but which are efficient in cases of interest. Of course, the inherent complexity of winner determination (i.e. the fact that the problem is NP-hard) means that there will always be cases where the algorithm runs in exponential time. But, remembering that NP-hardness is a *worst case* result, it may be that an algorithm has acceptable performance in cases of interest.

- Second, we can forget about optimality, and try to find techniques that are always efficient (i.e. don't require much computational effort) but that return 'reasonable' allocations, i.e. allocations that are not too far from the optimal. Again, we can identify two possible approaches:

  – use *heuristics* – 'rules of thumb' which seem to work well with practical examples, but for which we have no guarantee of good performance

  HEURISTIC WINNER DETERMINATION

  – use *approximation algorithms* – algorithms which are guaranteed to give us an answer in reasonable (polynomial) time, and which are guaranteed to give us an answer that is within some known bound of the optimal answer.

  APPROXIMATE WINNER DETERMINATION

With respect to optimal winner determination, a typical approach is to apply techniques for relevant combinatorial optimization problems. One popular approach is to formulate winner determination as an *integer linear programming* problem [Andersson et al., 2000]. In general, an integer linear program has the following structure. We are given some function f over integer variables $x_1, \ldots, x_k$, and we want to find values for the variables $x_1, \ldots, x_k$ that maximize the value of the function f. However, we are also given constraints on the values of the variables. For example, we might have one constraint, call it φ_1, which says that $x_1 + x_2 < x_3$, and another, call it φ_2, which says that $x_4 < 10$, and so on. So, we want to find values for variables $x_1, \ldots, x_k$ that maximize the function f subject to the constraints $\varphi_1, \varphi_2, \ldots, \varphi_l$. The function f is usually called the *objective function*. The 'linear' in the name 'linear integer programming' refers to the fact that all the terms used in functions and constraints must be linear functions. For example, $2x$ would be an allowable expression in a linear integer program, because it is a linear function of x: when you draw the graph of $y = 2x$ it is a straight line. In contrast, x^2 would not be allowable because it is not a linear function of x.

INTEGER LINEAR PROGRAMMING

OBJECTIVE FUNCTION

Usually, a linear integer program of the type discussed above is written in the following form:

$$\begin{aligned} \text{max imize} \quad & f(x_1, \ldots, x_k) \\ \text{subject to contraints} \quad & \varphi_1(x_1, \ldots, x_k) \\ & \varphi_2(x_1, \ldots, x_k) \\ & \vdots \\ & \vdots \end{aligned}$$

$$\varphi_1(x_1, \ldots, x_k)$$

where f is the function we want to maximize, and φ_1, φ_2, φ_l are the constraints on variables x_1, …, x_k.

Solving linear integer programs is, in general, computationally complex: it is very easy to see that it is NP-hard. However, much effort has been devoted to developing efficient algorithms for solving such problems, and a number of software tools for such problems are available.

We will now see how winner determination can be encoded as an integer linear program (the particular formulation here is from [Blumrosen and Nisan, 2007, p. 276]). To start with, we are given our set Z of goods, the set $Ag = \{1, \ldots, n\}$ of agents, and valuation functions v_1, …, v_n, one valuation function v_i for each agent $i \in Ag$. To create an integer linear program, we introduce variables $x_{i,Z}$ where $x_{i,Z}$ takes the value 1 if agent i is allocated the bundle $Z \subseteq Z$, and takes the value 0 otherwise. (You will notice that we are going to need a lot of such variables!)

$$\begin{aligned}
&\text{maximize} && \sum_{i \in Ag, Z \subseteq \mathcal{Z}} x_{i,Z} v_i(Z) \\
&\text{subject to constraints} && \sum_{i \in Ag, Z \subseteq \mathcal{Z} | z \in Z} x_{i,Z} \leq 1 \quad \text{for all } z \in \mathcal{Z} \\
& && \sum_{Z \subseteq \mathcal{Z}} x_{i,Z} \leq 1 \quad \text{for all } i \in Ag \\
& && x_{i,Z} \geq 0 \quad \text{for all } i \in Ag, Z \subseteq \mathcal{Z}.
\end{aligned}$$

Consider each part of this integer linear program:

- The objective function says that we are trying to find an allocation defined by the variables $x_{i,Z}$ that maximizes the sum of the corresponding $v_i(Z)$ values.
- The first constraint says that we don't allocate any good more than once. It expresses this constraint by saying that no more than one of the variables $x_{i,Z}$ corresponding to bundles Z in which item $j \in Z$ is a member can be true.
- The second constraint says that each agent is allocated no more than one bundle of goods.
- The final constraint simply says that the $x_{i,Z}$ values must be greater than or equal to 0. Together with the above constraints, this ensures that each variable $x_{i,Z}$ is either 0 or 1.

The main problem with this formulation is not the inherent complexity of the constraints themselves, but rather the number of variables (and hence the size of the integer linear program) that are generated. Despite this difficulty, the approach works surprisingly well in many cases [Andersson et al., 2000]. Many other approaches to exact, optimal winner determination have been investigated (e.g. see [Sandholm, 2002]).

With respect to approaches that do not guarantee an optimal result, the first thing we might look for is an algorithm that is guaranteed to terminate quickly, with a result that is guaranteed to be within some known (and hopefully reasonable) distance from the optimal solution. Such algorithms are called *approximation algorithms* [Ausiello et al., 1999].

Unfortunately, it is highly unlikely that winner determination can be approximated to within a small degree in this way [Lehmann et al., 2006, pp. 309–310]. This leaves heuristic approaches. Here, again, a wide number of approaches have been developed, for example, based on 'greedy' allocations, where we start by simply trying to find the largest bid we can satisfy, then the next largest, and so on [Sandholm, 2002, pp. 11–12].

14.3.3 The VCG mechanism

We saw above that naive mechanisms for combinatorial auctions are easy to strategically manipulate. In this section, we will see an ingenious way to construct combinatorial auctions that are not prone to manipulation in this way.

First, let us step back a little, and consider the problem that we are concerned with. We are in a combinatorial auction setting, so we have some set of goods Z, and some set of agents $Ag = \{1, \ldots, n\}$ with valuation functions $v_i : \mathbf{2}^Z \rightarrow \mathbb{R}$. We want a mechanism – an auction – that will have the property that if every participant acts rationally, then the outcome that is selected will maximize social welfare (i.e. will maximize the *sw*(…) function discussed above). If we knew the agents' *actual* valuations, this would be easy – we could directly apply the *sw* (…) function. But we don't have access to actual valuations: this is private information. So, the approach we will use is as follows. We design a mechanism so that the rational course of action is for an agent to reveal its true valuation to the mechanism. If everybody acts rationally, then the mechanism gets everybody's true valuation, and is able to find the allocation that maximizes social welfare. Of course, we saw earlier in this chapter that some auctions for individual goods – second-price sealed bid auctions, also called Vickrey auctions – have this property. The mechanism that we introduce is in fact a generalization of Vickrey's auction. The term *incentive compatible* is used to refer to a mechanism such as an auction in which telling the truth in this sense is the dominant strategy.

INCENTIVE COMPATIBLE

We need some additional terminology and notation. First, we will introduce an 'indifferent' valuation function: v^0 will be the valuation function that is completely indifferent about all possible sets of goods. Formally:

$$v^0(Z) = 0 \quad \text{for all } Z \subseteq Z.$$

Next, where $i \in Ag$ is an agent and $Z_1, \ldots, Z_n$ is an allocation of goods Z to agents in Ag, then $sw_{-i}(Z_1, \ldots, Z_n)$ will denote the social welfare of all the agents *except* agent i that is obtained from the allocation $Z_1, \ldots, Z_n$. Formally:

$$sw_{-i}(Z_1, \ldots, Z_n) = \sum_{j \in Ag : j \neq i} v_j(Z_j).$$

Thus $sw_{-i}(Z_1, \ldots, Z_n)$ is a measure of how well everybody *except* agent i does from $Z_1, \ldots, Z_n$.

Now, we can describe the auction mechanism. Formally, the auction is known as a *Vickrey–Clarke–Groves mechanism* (VCG mechanism), after those who invented its components. The VCG mechanism is as follows:

VCG MECHANISM

1. Every agent $i \in Ag$ simultaneously declares a valuation function v_i. (As before, we use $\hat{v}_i$ to emphasize that agent i can declare any valuation function it wants: its valuation function is private, and the mechanism has no way of telling whether or not $\hat{v}_i = v_i$.)

2. The mechanism computes the allocation $Z_1^*, \ldots, Z_n^*$ that maximizes the social welfare according to declared valuation functions $\hat{v}_i, \ldots, \hat{v}_n$.

   Formally, the mechanism computes:

   $$Z_1^*, \ldots, Z_n^* = \arg \max_{(Z_1, \ldots, Z_n) \in alloc(Z,Ag)} sw(Z_1, \ldots, Z_n, \hat{v}_1, \ldots, \hat{v}_i, \ldots, \hat{v}_n).$$

   The allocation $Z_1^*, \ldots, Z_n^*$ is chosen.

3. Every agent $i \in Ag$ pays to the mechanism an amount p_i. Intuitively, the amount p_i that agent i pays is 'compensation' to other agents for the total amount of utility that they lose by agent i participating. Think of it this way: if agent i participates and, as a consequence, is allocated some scarce resource, then i is denying some utility to other participants, who might have been allocated the resource instead. How do we figure out exactly how much this compensation is? It is the difference in social welfare to agents apart from i between the outcome $Z_1', \ldots, Z_n'$ that would have been chosen had i not participated, and the social welfare to agents apart from the allocation $Z_1^*, \ldots, Z_n^*$ i that is chosen when i participates.

Formally, let $Z_1', \ldots, Z_n'$ denote the allocation that would have been chosen had agent $i \in Ag$ declared valuation $\hat{v}_i = v^0$ and every other agent had made the same declaration:

$$Z_1', \ldots, Z_n' = \arg \max_{(Z_1, \ldots, Z_n) \in alloc(Z,Ag)} sw(Z_1, \ldots, Z_n, \hat{v}_1, \ldots, v^0, \ldots, \hat{v}_n)$$

Then the value p_i is:

$$p_i = sw_{-i}(Z_1', \ldots, Z_n', \hat{v}_1, \ldots, v^0, \ldots \hat{v}_n) - sw_{-i}(Z_1^*, \ldots, Z_n^*, \hat{v}_1, \ldots, \hat{v}_i, \ldots \hat{v}_n).$$

Now, the crucial point about this mechanism is the following: *no agent can benefit by declaring anything other than its true valuation*. That is, the VCG mechanism is incentive compatible. To see this, think about when the VCG mechanism is applied to the case where there is just a single good to allocate, i.e. Z is a singleton. Now, in this case, *the VCG mechanism is exactly the Vickrey auction that we described earlier!* This is because the only agent i that pays anything will be the one with the highest valuation; and had i not participated, the good would have been allocated to the agent that had the second highest valuation. The amount p_i that i would have to pay would be the amount of the second highest valuation. Thus in the case of a single good, the VCG mechanism reduces to the Vickrey mechanism, and is thus incentive compatible by the argument we saw earlier in this chapter. In other words, the VCG mechanism is a generalization of the Vickrey auction.

We thus have a mechanism that, assuming everybody plays their dominant strategy, finds the social welfare maximizing allocation. We can say that *social welfare maximization can be implemented in dominant strategies* in combinatorial auctions. Of course, dominant strategies

implemented in dominant strategies in combinatorial auctions. Of course, dominant strategies are only one of the solution concepts that we saw in Chapter 12; there are others. We can thus think of *mechanism design* problems where the relevant solution concept is Nash equilibria, and so on. In game theory, this is sometimes called *implementation theory* [Osborne and Rubinstein, 1994, pp. 177–196].

MECHANISM DESIGN

IMPLEMENTATION THEORY

What about the computational aspects of the VCG mechanism? It should be easy to see that, since the whole mechanism depends on computing the function *sw* (...), the mechanism is computationally very complex. We argued earlier that computing *sw*(...) is NP-hard, and so it is easy to see that computing payments with the VCG mechanism is also NP-hard. Much effort has gone into dealing with this complexity [Lavi, 2007; Müuller, 2006].

I hope you will agree that, despite the obvious computational difficulties surrounding it, the VCG mechanism is very elegant in what it does and how it does it. Researchers in the area of *algorithmic game theory* certainly do, as this has rapidly grown to be a very important area in computer science research [Nisan et al., 2007].

ALGORITHMIC GAME THEORY

14.4 Auctions in Practice

We conclude this chapter by looking at some examples of online auctions, and software agents to participate in these.

14.4.1 Online auctions

A highly active related area of work is *auction bots:* agents that can run, and participate in, online auctions for goods. A well-known example is the Kasbah system [Chavez and Maes, 1996]. The aim of Kasbah was to develop a web-based system in which users could create agents to buy and sell goods on their behalf. In Kasbah, a user can set three parameters for selling agents:

AUCTION BOTS

- desired date to sell the good by
- desired price to sell at
- minimum price to sell at.

Selling agents in Kasbah start by offering the good at the desired price, and as the deadline approaches, this price is systematically reduced to the minimum price fixed by the seller. The user can specify the 'decay' function used to determine the current offer price. Initially, three choices of decay function were offered: linear, quadratic, and cubic decay. The user was always asked to confirm sales, giving them the ultimate right of veto over the behaviour of the agent.

As with selling agents, various parameters could be fixed for buying agents: the date to buy the item by, the desired price, and the maximum price. Again, the user could specify the

'growth' function of price over time.

Agents in Kasbah operate in a *marketplace*. The marketplace manages a number of ongoing auctions. When a buyer or seller enters the marketplace, Kasbah matches up requests for goods against goods on sale, and puts buyers and sellers in touch with one another.

ELECTRONIC MARKETPLACE

Kasbah is loosely based on auctions such as those run by eBay and similar online auction houses [eBay, 2001]. Such auctions are big business: by 2004, it was estimated that US$ 60 billion per annum was being spent in online auction houses [Dixit and Skeath, 2004, p. 556]. For the most part, goods for sale on such auction houses are private value 'collectibles', rather than common value goods. Typically, the seller of the good is uncertain of its value, and the global nature of sites such as eBay maximizes their chance of finding an appropriate buyer. Sites such as eBay offer simple *proxy bidding* functionality, enabling bids to be placed automatically according to simple guidelines. Online auction sites use a range of different auction types, but English auctions are the most prevalent. Sites such as eBay add the following twist to English auctions: they introduce *deadlines*, so that the winner of the auction is the one who has submitted the highest bid above a reservation price by the deadline. A common behaviour in such auctions is called *sniping*. This is where bidders try to submit bids at the last possible moment. Sniping is profitable because it avoids revealing information about your valuation of a good, which could be exploited by others.

PROXY BIDDING

SNIPING

The *Spanish Fishmarket* is another example of an online auction system [Rodríguez et al., 1997]. Based on a real fishmarket that takes place in the town of Blanes in northern Spain, the fishmarket system provides similar facilities to Kasbah, but is specifically modelled on the auction protocol used in Blanes.

14.4.2 Adwords auctions

One use of auctions that has attracted a great deal of attention in recent years is the automated selling of advertising space on Web search engines using auctions. We take Google's 'AdWords' framework as an example [Google, 2008], although other search engines offer similar frameworks. The way it works is as follows. When you query a search engine such as Google, you do this by typing in some keywords (e.g. 'liverpool fc' would be a search for a Liverpool-based football club). The task of the search engine is to try to return to you a list of websites that seem most closely related to the terms that you typed, and of course different search engines use different technologies to try to give the best mapping from search terms to results. However, to earn revenue, web search sites typically market advertising space associated with search terms. For example, imagine you are a company that sells UK football memorabilia; you might be willing to pay to have your advert (or at least a link to your website) shown to anybody who searches for 'liverpool fc'. Google and other companies sell such rights using auctions. Google permits advertisers to participate in their 'AdWords' auctions by submitting bids; for example, your company might bid 40 cents for the right for your website to be listed alongside the "liverpool fc" search terms. Advertisers can set maximum bids, and can set limits on how much they want to spend. The auctions take place

maximum bids, and can set limits on how much they want to spend. The auctions take place every time a search is submitted (so of course advertisers submit bids in advance). Typically, more than one advertiser will appear alongside any given search, and both bidding price and relevance/quality is used to determine the order in which advertisers are listed. The exact auction type used is typically a variant of the Vickrey mechanism.

AdWords auctions are a fascinating example of how automated trading techniques have created an entirely new industry. The sums involved are very large: about 85% of Google's US$4.1 billion revenues in 2005 were derived from such mechanisms. See [Lahaie et al., 2007] for an introduction to the technical details of sponsored search auctions and further references.

14.4.3 The trading agent competition

We conclude this chapter by looking at the *trading agent competition* (TAC) [Wellman et al., 2007]. As its name suggests, the TAC is a competition, organized with the goal of furthering research in the development of agents that are capable of carrying out sophisticated automated transactions in something like real-world conditions. The idea of TAC is to submit a software agent for a simulated environment in which the agent must assemble travel packages for clients. The package consists of flights, hotels, and entertainment. Flights are sold to agents in auctions at regular intervals by an agent called TACAir; a continuous supply of flights is made available to the agents, but the pricing policy for these flights is governed by information that is private to TACAir. This motivates agents to try to model TACAir and predict the relevant price fluctuations. Accommodation is provided by two hotels, each providing 16 rooms per night; one of the hotels is cheap, hostel-style accommodation, and the other is an expensive premium hotel. Rooms are allocated through a series of ascending price auctions. To avoid buyers waiting until the last minute to submit their bids, auctions for rooms are designed to close at unpredictable times. Finally, three types of entertainment are provided, traded through a series of *continuous double auctions*, one for each type of entertainment on each day. Each participant is initially presented with a collection of tickets that it can buy or sell at its discretion. 'Double auction' in this setting means that participants are both buyers and sellers of goods (they can auction off tickets that they have previously obtained), while 'continuous' here means that the auctions take place continually (in something like real time).

TAC

CONTINUOUS DOUBLE AUCTION

The preferences of clients in the TAC are modelled by three parameters [Wellman et al., 2007, p. 13]:

- the ideal arrival and departure dates
- a value which indicates how much the agent values a premium hotel
- values indicating how much the client values each type of entertainment.

Given a specific trip package (consisting of flight, accommodation, and entertainment), the value of the package to the agent can be computed from these parameters via a simple equation [Wellman et al., 2007, p.14]. The utility of a trip to a client is the difference between the value of the trip package and the amount paid for it. The winner of the competition is the

agent that maximizes the utility for its clients.

TAC games are played over the Internet, with participant agents communicating via an agreed protocol, available to all participants as a free Java API. No restrictions are placed on the computing power of entrants, although restrictions are placed on what is permitted with agents at run time (e.g. no human intervention). The actual design of agents to participate in a TAC game is a complex issue, way beyond the scope of this book: see [Wellman et al., 2007] for a thorough discussion.

Notes and Further Reading

[Krishna, 2002] is the most comprehensive single-author introduction to auction theory that I am aware of, although it is mathematically fairly intense, and does not address computational issues. [Cramton et al., 2006] is a comprehensive collection of papers on computational aspects of auction theory, while [Nisan et al., 2007] is a very useful collection of papers on the more general (but very closely related) topic of algorithmic mechanism design; both of these books were heavily consulted when writing this chapter. [Milgrom, 2004] is an introduction to auction theory, put into context by one of the designers of the US spectrum auctions. An informal overview of the UK spectrum auctions is [Binmore and Klemperer, 2002]. [Chevaleyre et al., 2006] is a very readable survey of work on the general issue of resource allocation in multiagent systems, covering auctions, as well as some material (e.g. social choice) covered elsewhere in this book.

Class reading: [Sandholm, 2007]. This gives a detailed case study of a successful company operating in the area of computational combinatorial auctions for industrial procurement.

Figure 14.1: Mind map for this chapter.